Menoufia University

Faculty of Electronic Engineering in Menouf

Department of Computer Science & Engineering

Virtual Office

A Spatial, All-in-One Team Collaboration Platform

Graduation Project ▪ Final Documentation

Supervised by

Prof. Dr. Nermeen Abdelwahab El-Behnsawy

Team

Abdelrahman Sersawy
15210161

Ahmed Mousa
1523051

Khaled Qenawy
15210114

Khalid Labib
15210113

Rokaya Othman
15210124

Nadine Mohsen
15210356

Academic Year 2025 / 2026 ▪ Menouf, Egypt

Abstract

The shift toward remote and hybrid work has scattered the modern workplace across a dozen disconnected tools: one application for video meetings, another for chat, others for tasks, calendars, and file sharing. This fragmentation erodes the two qualities that make a physical office productive — *presence* (knowing who is around and available) and *spontaneity* (the ability to walk up to a colleague and start talking). Virtual Office is a graduation project that addresses this gap by delivering a single, spatial, all-in-one collaboration platform: a digital headquarters where teams appear as avatars in an animated 2D office and where messaging, meetings, tasks, scheduling, and workspace administration live under one roof.

The platform is engineered as three coordinated front-ends over a distributed backend. An immersive **2D virtual office** — built with the Phaser game engine on the client and an authoritative Colyseus multiplayer server — renders avatars that walk around a tile-mapped office, sit at desks, gather at whiteboards, and start proximity-based voice, video, and screen-sharing sessions the moment they move close to one another. A **React web client** provides a full Slack-style collaboration surface — workspaces, channels, direct messages, threads, room calls (powered by Agora real-time media), and a live notification centre. A **Flutter mobile client** brings the messaging experience to phones, complete with optimistic delivery, read receipts, threads, and cross-state push notifications. Underneath, a **microservices backend** of nine cooperating Spring Boot services — gateway, user, workspace, room, chat, tasks, calendar, notifications, and a shared library — handles identity, spatial state, real-time messaging, and event-driven side effects, coordinated through an Nginx gateway, per-service databases (PostgreSQL, MongoDB, Redis), and a RabbitMQ message bus.

This book documents the complete engineering of the system: the problem it solves and the requirements it targets; the microservices architecture and the cross-cutting patterns that hold it together (token-based WebSocket authentication, the `{action, payload}` real-time envelope, event-driven service decoupling, and store-the-foreign-key service boundaries); the internal design of each backend service, including its data model, REST and WebSocket APIs, and message flows; the real-time multiplayer model of the 2D office; and the implementation of the web and mobile clients. It closes with the project's testing approach, an honest evaluation of what was achieved, and a roadmap for future work.

Keywords: virtual office, spatial collaboration, microservices, Spring Boot, real-time systems, WebSocket, STOMP, WebRTC, Phaser, Colyseus, React, Flutter, event-driven architecture.

Contents

Abstract i

1 Introduction 1
- 1.1 Overview . . . 1
- 1.2 Motivation . . . 1
- 1.3 Problem Statement . . . 2
- 1.4 Objectives . . . 2
- 1.5 Scope of the Project . . . 3
- 1.6 Main Contributions . . . 3
- 1.7 Organisation of the Book . . . 4

2 Background and Related Work 5
- 2.1 The Virtual Office Concept . . . 5
- 2.2 Enabling Technologies . . . 5
  - 2.2.1 Real-time Transport: WebSockets and STOMP . . . 5
  - 2.2.2 Peer-to-peer Media: WebRTC, PeerJS, and Agora . . . 6
  - 2.2.3 Game Engines and Authoritative Servers . . . 6
  - 2.2.4 Microservices Architecture . . . 7
- 2.3 Related Work and Existing Tools . . . 7
- 2.4 The Gap and This Project's Position . . . 8

3 Requirements Analysis 9
- 3.1 Actors . . . 9
- 3.2 Functional Requirements . . . 9
  - 3.2.1 Identity and Accounts (User Service) . . . 10
  - 3.2.2 Workspaces, Desks, and Teams (Workspace Service) . . . 10
  - 3.2.3 Messaging (Chat Service) . . . 10
  - 3.2.4 Meeting Rooms and Proximity Voice (Room Service) . . . 11
  - 3.2.5 2D Virtual Office (SkyOffice) . . . 11
  - 3.2.6 Notifications (Notification Service) . . . 11
  - 3.2.7 Cross-cutting Client Requirements . . . 11
- 3.3 Non-functional Requirements . . . 11
- 3.4 Use-Case Model . . . 13
  - 3.4.1 Detailed Use Cases . . . 13
- 3.5 User Stories . . . 13
- 3.6 Domain Model . . . 15

4 System Architecture 16
- 4.1 Architectural Style . . . 16
- 4.2 System Overview . . . 16
- 4.3 Service Catalogue . . . 16
- 4.4 The API Gateway and Security Model . . . 18
- 4.5 Communication Patterns . . . 19
  - 4.5.1 Synchronous: internal REST . . . 19
  - 4.5.2 Asynchronous: the RabbitMQ event bus . . . 19

4.6 The Real-time Layer . . . 20
4.6.1 Ticket-based WebSocket authentication . . . 20
4.6.2 The `{action, payload}` envelope . . . 20
4.7 Polyglot Persistence . . . 21
4.8 Technology Stack Summary . . . 21
4.9 Deployment View . . . 21

5 Backend Implementation **23**
5.1 Gateway API . . . 23
5.2 User Service . . . 24
5.3 Workspace Service . . . 25
5.4 Chat Service . . . 26
5.5 Room Service . . . 28
5.6 Notification Service . . . 29
5.7 The Unimplemented Modules . . . 29
5.8 Cross-service Consistency in Practice . . . 30

6 The 2D Virtual Office **31**
6.1 Overview and Structure . . . 31
6.2 The Shared Type Contract . . . 31
6.3 The Authoritative Server . . . 32
6.4 The Client . . . 32
6.4.1 Scenes . . . 32
6.4.2 Characters . . . 33
6.4.3 The React and Redux overlay . . . 33
6.5 The Multiplayer Synchronisation Model . . . 33
6.6 Interactive Objects . . . 34
6.7 Proximity Communication . . . 34
6.8 Integration with the Wider Platform . . . 35

7 The Web Application **36**
7.1 Overview and Technology Stack . . . 36
7.2 Application Structure and Routing . . . 36
7.3 Authentication and the API Layer . . . 37
7.4 The Real-time Layer: Three STOMP Connections . . . 37
7.5 Voice and Video with Agora . . . 38
7.6 Feature Tour . . . 38
7.7 Design Observations . . . 39

8 The Mobile Application **40**
8.1 Overview and Scope . . . 40
8.2 Architecture . . . 40
8.3 State Management with Provider . . . 41
8.4 Networking: A Dual-Transport Repository . . . 41
8.5 Identity on Mobile . . . 42
8.6 Notifications Across App States . . . 42
8.7 Data Models and Bootstrapping . . . 42
8.8 Engineering Highlights . . . 42

9 Testing, Deployment, and Evaluation **44**
9.1 Testing Strategy . . . 44
9.2 Deployment and the Demonstration Stack . . . 44
9.3 Observability: Metrics and Dashboards . . . 45
9.3.1 The Metrics Pipeline . . . 45
9.3.2 What the Dashboards Show . . . 46
9.4 Evaluation against Requirements . . . 47
9.5 Non-functional Evaluation . . . 47
9.6 Limitations . . . 48

10 Conclusion and Future Work **49**
10.1 Summary . . . 49
10.2 Achievements . . . 49
10.3 Lessons Learned . . . 50
10.4 Future Work . . . 50
10.5 Closing Remarks . . . 51

List of Figures

3.1 Top-level use-case diagram for Virtual Office. 13
3.2 Conceptual domain model of Virtual Office (cross-service view). 15

4.1 Virtual Office system architecture. Solid grey arrows: request/response and storage; dashed green: asynchronous events over RabbitMQ; dotted blue: synchronous internal role lookups; dashed boxes: unimplemented scaffolds. . 17
4.2 End-to-end authentication and identity-propagation flow. 19
4.3 Ticket-based WebSocket handshake, used identically by the chat, room, and notification services. 20

5.1 Workspace-service core relationships. A Desk joins a User to a Workspace and carries the workspace role. 26
5.2 Message-send flow in the chat service. 27

6.1 Movement synchronisation: local prediction, authoritative relay, remote interpolation. Only changed fields cross the wire. 34

7.1 Joining a room call: the room service mints the Agora token, media flows client-to-SFU. 38

8.1 Mobile clean-architecture layering; data flows WS/REST → repository → providers → widgets. 40
8.2 Optimistic send with WebSocket-first, REST-fallback, and echo reconciliation. 42

9.1 The metrics pipeline: each implemented service exposes a Prometheus endpoint; one Prometheus server scrapes all five and stores the series, and Grafana queries it to render the HTTP and datastore dashboards. 45

List of Tables

2.1 Comparison of Virtual Office with representative existing tools. 8

3.1 System actors. 9
3.2 Non-functional requirements. 12
3.3 Use case UC-05: Send a message to a channel. 14
3.4 Use case UC-08: Proximity voice/video in the 2D office. 14

4.1 Backend service catalogue. 18
4.2 Event flows over RabbitMQ. All use durable direct exchanges, JSON payloads, and consumer-side retry with exponential backoff. 19
4.3 Technology stack by tier. 21

5.1 Gateway routing rules. 23
5.2 User service entities. 24
5.3 User service REST API. 25
5.4 Principal workspace-service entities (PostgreSQL). 26
5.5 Chat service collections. 27

6.1 Structure of the 2D office component. 31

7.1 Web client routes. 36
7.2 Development proxy: the frontend's view of the backend. 37

9.1 Evaluation of functional requirement groups. 47

CHAPTER 1

Introduction

1.1 Overview

Work no longer happens in one place. Over the last decade, and dramatically so after 2020, teams have moved from a shared physical office to a distributed arrangement in which colleagues may be in different homes, cities, or time zones. The tools that grew up to support this transition are powerful but *disjoint*: a team may hold meetings in one application, chat in a second, track tasks in a third, keep its calendar in a fourth, and store files in a fifth. Each tool is a silo with its own identity, its own notifications, and its own mental model. The result is a workday spent switching between browser tabs and losing the two things a real office provides for free: an ambient sense of *who is present and available*, and the *spontaneous* conversations that begin simply because two people happen to be near each other.

Virtual Office is a graduation project that responds to this problem with a single idea: give a distributed team a *place* rather than a collection of tools. That place is a two-dimensional, game-like office — a digital headquarters — in which every member is represented by an animated avatar. Users walk around a tile-mapped floor, see their colleagues moving in real time, sit down at desks, gather around whiteboards, and slip into meeting corners. When two avatars move close together, a proximity-based voice and video call opens automatically, just as a hallway conversation would; when they move apart, it ends. Around this spatial core, Virtual Office folds in the rest of the collaboration stack — persistent chat channels and direct messages, threaded discussions, task assignment, calendar scheduling, room-based meetings, and a unified notification system — so that the entire working day can happen inside one product.

Delivering this experience is a substantial distributed-systems undertaking. The platform comprises an animated multiplayer world, a full-featured web client, a native-quality mobile client, and a backend of nine cooperating microservices, all bound together by real-time messaging over WebSockets, event-driven communication over a message bus, and a shared identity model enforced at the gateway. This book is a complete engineering account of that system.

1.2 Motivation

The motivation for Virtual Office rests on three observations about modern distributed work.

Tool fragmentation is expensive. The average knowledge worker now moves between many single-purpose applications during a day. Each context switch carries a cognitive cost, and each additional tool multiplies the surfaces on which information can be lost. Consolidating presence, communication, tasks, and scheduling into one platform removes friction and gives a team a single source of truth for "what is happening right now."

Video calls kill spontaneity. Scheduled video meetings are excellent for planned discussion but hostile to the unplanned. In a physical office, most valuable communication

is incidental — a question asked in passing, an overheard problem, a quick huddle at a desk. Grid-of-faces video conferencing has no equivalent: you cannot "walk past" someone in a Zoom call. A spatial interface, where being near a colleague is enough to start talking, restores this incidental channel.

Presence is a first-class need. Knowing who is online, who is busy, and who is deep in focus is information a physical office broadcasts continuously and effortlessly. Most remote tools reduce this to a coloured dot. A spatial office makes presence *legible*: you can literally see where people are, whether they are at their desk, in a meeting room, or gathered with others.

Commercial products such as Gather and other "metaverse office" tools have validated that a spatial approach resonates with users. VIRTUAL OFFICE is motivated by the goal of building such a platform from first principles — not as a thin demo, but as a properly architected, multi-client, microservices-based system — and, in doing so, of learning and demonstrating the engineering that a real real-time collaboration product requires.

1.3 Problem Statement

The project sets out to answer a single engineering question:

Problem Statement

How can a distributed team be given a unified digital workplace that reproduces the *presence* and *spontaneity* of a physical office, while consolidating the fragmented tools of remote work — messaging, meetings, tasks, scheduling, and workspace administration — into one coherent, real-time, multi-client platform?

Answering it well requires solving several sub-problems that recur throughout this book:

1. **Real-time spatial state.** Many users must see each other move around a shared world with low latency and without the network cost of broadcasting every pixel of movement to everyone.
2. **Proximity-driven communication.** Voice, video, and screen sharing must start and stop based on where avatars are, and must scale without routing media through the application's own servers.
3. **Reliable messaging.** Chat must deliver instantly, survive disconnections, avoid duplicates, and preserve history — across both a web and a mobile client.
4. **A coherent identity and permission model** that is enforced once, at the edge, and trusted by every internal service.
5. **Loose coupling between services** so that features such as notifications and room–chat binding can evolve independently and survive partial outages.

1.4 Objectives

The concrete objectives of the VIRTUAL OFFICE project are:

1. To design and implement an **immersive 2D virtual office** in which multiple users, represented as avatars, move and interact in real time within a shared, persistent space.
2. To provide **proximity-based real-time communication** — voice, video, and screen sharing — that begins and ends according to spatial closeness, using an external media

infrastructure so the platform's own servers never carry audio or video streams.

3. To build a **full-featured messaging system** supporting workspace channels, direct messages, threaded conversations, mentions, replies, edits, soft deletion, read receipts, and typing indicators, delivered over WebSockets with a REST fallback.
4. To implement a **workspace and identity layer** that models companies, teams, desks (user seats), roles, and invitations, and that issues and validates authentication centrally.
5. To add the surrounding **collaboration services** — task assignment and tracking, calendar scheduling, meeting rooms, and a unified notification centre (both e-mail and in-app).
6. To expose the platform through **three clients** — a 2D multiplayer world, a React web application, and a Flutter mobile application — that share the same backend.
7. To realise all of the above on a **microservices architecture** that demonstrates sound distributed-systems practice: an API gateway, per-service data ownership, event-driven decoupling, and consistent cross-cutting patterns for security and real-time communication.

1.5 Scope of the Project

Virtual Office is a large system, and it is important to state clearly what this book documents and where the boundaries lie.

In scope. The book covers the full breadth of the delivered system: the nine backend microservices and their data models, APIs, and message flows; the 2D multiplayer office (client and server); the React web client; and the Flutter mobile client. It covers the cross-cutting mechanisms that make the platform work as a whole — gateway authentication, token-based WebSocket handshakes, the event bus, and the real-time messaging protocol.

Boundaries and honest limitations. Virtual Office is a graduation project, and several parts of the system are at differing levels of maturity. As this book documents faithfully, some capabilities are implemented end-to-end (chat, workspaces, the 2D office, rooms), some are implemented on one client but not another (the mobile app implements only the chat domain), some backend services are more complete than others, and some features are deliberately left as "version 2" hardening (for example, server-side minting of media tokens, or role-based authorisation checks that are currently trusted to the client). Rather than gloss over these, the book records them as they are; they are valuable both as an accurate description of the system and as a guide to future work.

1.6 Main Contributions

The principal contributions of the project, and of this book, are:

- A complete, working design for a **spatial collaboration platform** that unifies a real-time 2D office with a full messaging, tasks, calendar, and workspace stack.
- A **nine-service microservices backend** built on Spring Boot, demonstrating an API gateway with centralised authentication, polyglot persistence (PostgreSQL, MongoDB, Redis), and event-driven decoupling over RabbitMQ.

- A set of **reusable real-time patterns** — ticket-based WebSocket authentication, a uniform `{action, payload}` broadcast envelope, and proximity-triggered peer-to-peer media — applied consistently across services and clients.
- Three **distinct client implementations** (2D game, web, mobile) over one backend, illustrating how a single platform can be surfaced through very different technologies.
- A thorough **engineering write-up** that can serve as a reference for building real-time, multi-client collaboration systems.

1.7 Organisation of the Book

The remainder of this book is organised as follows.

Chapter 2 surveys the background and related work: the concept of the virtual office, the enabling technologies (WebSockets, WebRTC, microservices, game engines), and a comparison with existing tools.

Chapter 3 presents the requirements analysis — actors, functional and non-functional requirements, user stories, and use-case and domain models expressed in UML.

Chapter 4 describes the overall system architecture: the microservices, the gateway, the data and messaging infrastructure, and the cross-cutting patterns that unify them.

Chapter 5 documents each backend microservice in depth — its responsibility, data model, REST and WebSocket API, and its event interactions with the rest of the system.

Chapter 6 details the 2D virtual office: its client/server split, the multiplayer synchronisation model, interactive objects, and the proximity-based communication stack.

Chapter 7 and Chapter 8 cover the web and mobile clients respectively, including their architecture, real-time integration, and feature sets.

Chapter 9 discusses the testing and evaluation approach, and Chapter 10 concludes with an assessment of the project and a roadmap for future work.

CHAPTER 2

Background and Related Work

This chapter situates Virtual Office in its technical and product context. It first explains the concept of the *virtual office* and why a spatial metaphor is well suited to remote collaboration. It then reviews the enabling technologies that make such a system possible — real-time transport, peer-to-peer media, game engines, and microservices — so that the design decisions in later chapters can be read against a shared vocabulary. Finally, it compares Virtual Office with existing tools and identifies the gap it aims to fill.

2.1 The Virtual Office Concept

A *virtual office* (sometimes called a spatial collaboration platform or a "metaverse office") is a persistent, shared, two- or three-dimensional environment in which remote team members are represented by avatars. Rather than joining a meeting, a user *enters a space*. Presence and communication are governed by *spatial proximity*: two users who move their avatars close to one another can talk, exactly as they would in a physical room, and their conversation is private to the people nearby.

This model differs fundamentally from conventional remote-collaboration tools in three ways:

Continuous presence, not discrete sessions. In a video-conferencing tool, you are either in a call or not. In a virtual office, you are simply *present in the space* for as long as you keep it open, and others can see where you are and what you are doing. Communication is a continuous ambient possibility rather than a scheduled event.

Proximity as the routing rule. Who hears whom is determined by position. Walking toward a colleague opens a channel; walking away closes it. This reproduces the natural way conversations form and dissolve in a physical workplace, and it makes group boundaries (a meeting room, a huddle, a quiet corner) visible and intuitive.

Spatial memory and identity. People associate activities with places — "the meeting room," "my desk," "the whiteboard by the window." A persistent map gives a distributed team the same spatial anchors, which aids orientation, belonging, and informal coordination.

The trade-off is engineering complexity. A virtual office is, at its heart, a small *multiplayer game*: it must synchronise the positions of many moving avatars in real time, manage shared interactive objects, and start and stop live media according to where those avatars are — all while remaining responsive. The remainder of this book is, in large part, an account of how Virtual Office meets that complexity.

2.2 Enabling Technologies

2.2.1 Real-time Transport: WebSockets and STOMP

Ordinary web traffic follows a request–response pattern over HTTP: the client asks, the server answers, and the connection is (logically) done. This is a poor fit for collaboration, where the server must *push* events — a new message, a colleague moving, a notification — to

the client without being asked. **WebSockets** solve this by upgrading an HTTP connection into a persistent, bidirectional channel over which either side can send frames at any time.

On top of raw WebSockets, Virtual Office uses **STOMP** (Simple Text Oriented Messaging Protocol), a lightweight publish/subscribe protocol. STOMP gives the raw socket a structure of *destinations*: clients `SUBSCRIBE` to topics such as `/topic/channel/42` and `SEND` messages to application destinations such as `/app/chat/send`, while the server broadcasts to subscribers or to a specific user's private queue. This maps cleanly onto the domain — a channel is a topic, a user's error stream is a personal queue — and it is supported natively by Spring on the server and by client libraries in JavaScript (`@stomp/stompjs`) and Dart (`stomp_dart_client`), both of which Virtual Office uses.

The WebSocket Authentication Problem

A browser cannot attach custom headers (such as an `Authorization` bearer token) to a WebSocket handshake. Virtual Office solves this uniformly across its real-time services with a *ticket* pattern: the client makes an authenticated REST call to obtain a short-lived, single-use ticket, then opens the socket with the ticket as a query parameter. This recurring pattern is described in detail in Chapter 4.

2.2.2 Peer-to-peer Media: WebRTC, PeerJS, and Agora

Carrying live audio and video is expensive and latency-sensitive. Routing every stream through the application's own servers would be costly and would not scale. The industry standard for browser media is **WebRTC**, which establishes direct, encrypted, peer-to-peer connections between clients for audio, video, and arbitrary data, using the application server only for the initial *signalling* (helping peers discover and describe each other).

Virtual Office uses two complementary media stacks:

- In the 2D office, proximity calls are built on **PeerJS**, a thin wrapper over WebRTC that assigns each peer an id and simplifies the call/answer handshake. The office reuses the multiplayer session id as the PeerJS id, so one identity threads both systems.
- In the web client's meeting rooms, media is handled by **Agora**, a commercial real-time-communication platform whose Selective Forwarding Unit (SFU) relays streams efficiently. Here the application server acts as a *token server*, minting short-lived, privilege-scoped access tokens so that only authorised users can join a room's media channel — while the media itself never touches the application's servers.

The distinction matters: peer-to-peer (PeerJS/WebRTC) is simple and free but scales poorly beyond small groups, whereas an SFU (Agora) scales to larger rooms at the cost of a third-party dependency. Virtual Office deliberately uses each where it fits.

2.2.3 Game Engines and Authoritative Servers

The animated office is a real game, and it is built with a real game engine. **Phaser** is a mature 2D HTML5 game framework providing sprite rendering, tile-map support, arcade physics (for collisions and overlap detection), animations, and camera control — everything needed to render a walkable, collision-aware office in the browser.

Multiplayer state is managed by **Colyseus**, an authoritative multiplayer server framework. "Authoritative" means the server holds the single source of truth for the shared world state; clients send their intentions and receive the resulting state. Colyseus's schema system serialises only the *changed fields* of the state as compact binary diffs, so moving one avatar does not require re-sending the entire world. This is the key to keeping many simultaneous

players responsive, and Chapter 6 examines the resulting synchronisation model, including client-side prediction and remote interpolation, in detail.

2.2.4 Microservices Architecture

Rather than a single large application (a *monolith*), Virtual Office's backend is decomposed into independent **microservices**, each owning one business capability (identity, chat, rooms, workspaces, and so on) and its own database. Services communicate through well-defined interfaces — synchronous REST for queries that need an immediate answer, and asynchronous events over a message broker for changes that other services should react to but need not wait for.

This style brings several benefits that recur through the book: each service can use the persistence technology that best fits its data (a theme called *polyglot persistence*); services can be developed, deployed, and scaled independently; and a failure in one service can be contained rather than taking down the whole system. It also introduces challenges — distributed data consistency, cross-service authentication, and operational complexity — whose solutions form much of Chapter 4. The supporting cast of technologies includes **Spring Boot** and **Spring Cloud Gateway** (the service and gateway framework), **RabbitMQ** (the message broker for events), and a mix of **PostgreSQL**, **MongoDB**, and **Redis** for storage.

2.3 Related Work and Existing Tools

Virtual Office sits at the intersection of two product categories — spatial/virtual offices and unified team-collaboration suites — and borrows from both. The most relevant points of comparison are summarised below.

Gather (gather.town). The best-known commercial virtual office: a 2D, game-like map with avatars and proximity video, aimed at remote teams and events. Gather validated the spatial model at scale. Virtual Office adopts the same core interaction (avatars, proximity media, interactive objects) but pairs it with a full, self-hosted collaboration backend rather than being purely a meeting space.

SkyOffice. An open-source Phaser + Colyseus virtual-office project. Virtual Office's 2D world is an adapted fork of SkyOffice, which provides a strong, well-architected base for avatar movement, interactive computers and whiteboards, and proximity communication. The contribution of Virtual Office is not the 2D engine in isolation but its integration into a broader, multi-client, microservices platform with real identity, workspaces, and persistent chat.

Slack and Microsoft Teams. The dominant unified-collaboration suites. They excel at persistent, searchable messaging (channels, DMs, threads) and integrate calls, files, and apps — but they are fundamentally *list-based*, not spatial: presence is a coloured dot and communication is a discrete call or message. Virtual Office's chat subsystem is deliberately modelled on the Slack workspace/channel/thread structure (as later chapters show), but it is embedded in a spatial world that these tools lack.

Discord. Combines persistent text channels with always-available voice "rooms" that users drop into and out of. Virtual Office's room service is explicitly Discord-like: a room bundles a persistent text channel with a joinable voice/video space. The difference again is the spatial layer and the workspace-centric organisation.

Zoom / Google Meet. Best-in-class scheduled video conferencing, but session-based and non-persistent: there is no shared space, no ambient presence, and no spontaneous

encounter. VIRTUAL OFFICE treats real-time media as one capability among many rather than the whole product.

Table 2.1 positions VIRTUAL OFFICE against these tools along the dimensions that matter for the problem stated in Chapter 1.

Table 2.1. Comparison of VIRTUAL OFFICE with representative existing tools.

Capability	**Gather**	**Slack / Teams**	**Discord**	**Zoom**	**Virtual Office**
Spatial 2D world	✓	—	—	—	✓
Proximity media	✓	—	partial	—	✓
Persistent channels	partial	✓	✓	—	✓
Threads & mentions	—	✓	partial	—	✓
Meeting rooms	✓	✓	✓	✓	✓
Tasks & calendar	—	✓	—	—	planned
Self-hosted backend	—	—	—	—	✓
Native mobile client	✓	✓	✓	✓	partial

2.4 The Gap and This Project's Position

Existing tools each solve part of the problem. Spatial offices deliver presence and spontaneity but are typically thin on persistent, structured collaboration and are closed, hosted products. Unified suites deliver excellent messaging and integration but are list-based and lack a sense of place. Video tools deliver media quality but nothing else.

VIRTUAL OFFICE occupies the intersection: a spatial, game-like office *and* a full, self-hosted collaboration backend — messaging, rooms, workspaces, and the scaffolding for tasks and calendars — surfaced through multiple clients. The engineering value of the project lies in showing how these very different concerns (a real-time multiplayer game, a Slack-style chat system, a microservices identity and workspace layer, and peer-to-peer media) can be integrated into one coherent platform. The chapters that follow describe that integration from requirements through architecture to implementation.

CHAPTER 3

Requirements Analysis

This chapter turns the motivation of Chapter 1 into a precise specification. It identifies the system's actors, enumerates its functional and non-functional requirements, expresses the key interactions as use cases (with UML diagrams and detailed descriptions), captures representative user stories, and presents the domain model as a class diagram. Because Virtual Office is a multi-service, multi-client system, the requirements are organised by *subsystem* so that they map cleanly onto the architecture and implementation chapters that follow.

3.1 Actors

An *actor* is any external entity that interacts with the system. Virtual Office has both human and external-system actors, summarised in Table 3.1.

Table 3.1. System actors.

Actor	Type	Description
Guest	Human	An unauthenticated visitor who can register, log in, or accept an invitation.
Member	Human	An authenticated user with a *desk* in a workspace; the primary actor. Can chat, join rooms, move around the 2D office, and manage their own profile.
Workspace Admin	Human	A member with the `ADMIN` role in a workspace; can manage teams, invite and remove members, and edit the office map layout.
Workspace Owner	Human	The creator of a workspace, holding the `OWNER` role; has all admin powers plus the ability to archive the workspace.
Colyseus Server	System	The 2D game server; synchronises avatar positions and pushes presence snapshots to the backend.
Agora Platform	System	External real-time media (SFU) that transports room and proximity audio/video.
Email/SMTP	System	External mail transport used to deliver transactional e-mails (sign-up, OTP, password reset).

The human actors form a role hierarchy: an Owner is an Admin is a Member is a (formerly) Guest. Authorisation throughout the system is expressed against the ranked workspace roles `GUEST < MEMBER < ADMIN < OWNER`.

3.2 Functional Requirements

Functional requirements are grouped by subsystem. Each is given a stable identifier (`FR-`*xx*) for traceability.

3.2.1 Identity and Accounts (User Service)

FR-01 A guest shall register with first name, last name, e-mail, phone number, and password; the system shall reject a duplicate e-mail.

FR-02 A registered user shall log in with e-mail and password and receive an authentication token valid for a bounded period.

FR-03 The system shall hash all passwords and one-time passwords before storage.

FR-04 A user shall request a one-time password (OTP) for password reset and verify it within a time limit.

FR-05 A user shall change their password by supplying the current one.

FR-06 A user shall view their profile and upload/retrieve a profile picture.

FR-07 The system shall emit an event on sign-up, OTP request, and password change so that a confirmation e-mail is sent.

3.2.2 Workspaces, Desks, and Teams (Workspace Service)

FR-08 A member shall create a workspace, becoming its owner and receiving an owner desk in the same transaction.

FR-09 A workspace shall be invite-only; an admin shall invite users by e-mail, and an invitation shall expire after a fixed period.

FR-10 A guest shall accept or decline an invitation via a tokenised link; acceptance shall create or reactivate the invitee's desk.

FR-11 A member shall view the workspace member directory and edit their own desk (profile, avatar, status, links, widgets, position).

FR-12 An admin shall create, rename, and delete teams and assign members to them.

FR-13 An admin shall edit the 2D map layout (tilesets, layers, zones, spawn points, map objects) with optimistic-concurrency protection.

FR-14 The system shall expose an internal session-configuration API that provides the game server with the full room boot payload (workspace, active desks, active map objects).

FR-15 The system shall accept avatar presence updates (online status, position) from the game server and resolve a member's workspace role on request.

3.2.3 Messaging (Chat Service)

FR-16 A member shall create and join group channels within a workspace and leave them.

FR-17 A member shall open a direct-message conversation with another user; the system shall guarantee a single DM per user pair.

FR-18 A member shall send text messages to a channel, DM, or thread and receive them in real time.

FR-19 A member shall reply to a message, mention other users, edit their own messages, and soft-delete messages.

FR-20 A member shall start a thread from a message and reply within it.

FR-21 The system shall track per-user read position and report unread counts for channels and threads.

FR-22 The system shall broadcast typing indicators.

FR-23 The system shall guarantee idempotent message delivery so that a retried send is not duplicated.

FR-24 The system shall create, update, and delete `ROOM`- and workspace-level channels in response to events from the room and workspace services.

3.2.4 Meeting Rooms and Proximity Voice (Room Service)

FR-25 A member shall create a voice/video room bound to a text channel, and add or remove room members.

FR-26 A member shall join a room and receive a media access token and the current participant list; joins shall respect a capacity limit.

FR-27 The system shall track live presence in a room (mute, camera, screen-share state) and broadcast join/leave/state changes in real time.

FR-28 The system shall compute proximity-based voice groups from avatar positions: users within a zone or within hearing range shall be assigned to a shared media channel.

3.2.5 2D Virtual Office (SkyOffice)

FR-29 A user shall appear as an avatar and move around a tile-mapped office with keyboard or on-screen joystick, colliding with walls and objects.

FR-30 A user shall see other users move in real time with smooth motion.

FR-31 A user shall sit on chairs, and interact with computers (screen-share stations) and whiteboards.

FR-32 Proximity between two avatars shall automatically start and stop a peer-to-peer video call; a designated area shall keep a call alive as a private meeting space.

FR-33 A user shall exchange in-world chat messages, shown both in a chat panel and as transient speech bubbles.

3.2.6 Notifications (Notification Service)

FR-34 The system shall consume domain events and deliver transactional e-mails from versioned HTML templates.

FR-35 The system shall persist in-app notifications and push them to online users in real time, backfilling for offline users.

FR-36 A user shall list notifications, see an unread count, mark items read (individually or all), and delete them.

FR-37 Notification delivery shall be idempotent with respect to a unique event identifier.

3.2.7 Cross-cutting Client Requirements

FR-38 The web client shall provide the full workspace, channel, DM, thread, room-call, and notification experience.

FR-39 The mobile client shall provide the messaging experience (channels, DMs, threads) with push notifications across foreground, background, and terminated states.

FR-40 All real-time clients shall authenticate their WebSocket connection using a single-use ticket and reconnect automatically.

3.3 Non-functional Requirements

Table 3.2. Non-functional requirements.

ID	Quality	Requirement
NFR-01	Performance	Avatar movement and chat delivery shall feel instantaneous; movement shall use client-side prediction and interpolation, and state sync shall transmit only changed fields.
NFR-02	Scalability	Media (audio/video/screen) shall never traverse the application servers; it shall use peer-to-peer or an external SFU so that server load is independent of media volume.
NFR-03	Security	Authentication shall be validated once at the gateway; internal endpoints shall be unreachable from the edge; client-supplied identity headers shall be stripped.
NFR-04	Reliability	Event consumers shall retry with exponential backoff and process events idempotently; a service outage shall degrade gracefully rather than cascade.
NFR-05	Consistency	Cross-service state (e.g. room-to-chat channel binding) shall be eventually consistent via durable events, with clients tolerating brief propagation delay.
NFR-06	Maintainability	Each service shall own its data and schema; services shall be independently buildable and deployable.
NFR-07	Portability	Each service shall ship with a container/compose definition for its dependencies to enable local and containerised deployment.
NFR-08	Usability	Clients shall support light and dark themes and responsive layouts across desktop and mobile form factors.
NFR-09	Observability	Services shall expose health endpoints and structured logs for monitoring.

3.4 Use-Case Model

Figure 3.1 gives the top-level use-case diagram, grouping use cases by subsystem and relating them to the actors of Section 3.1.

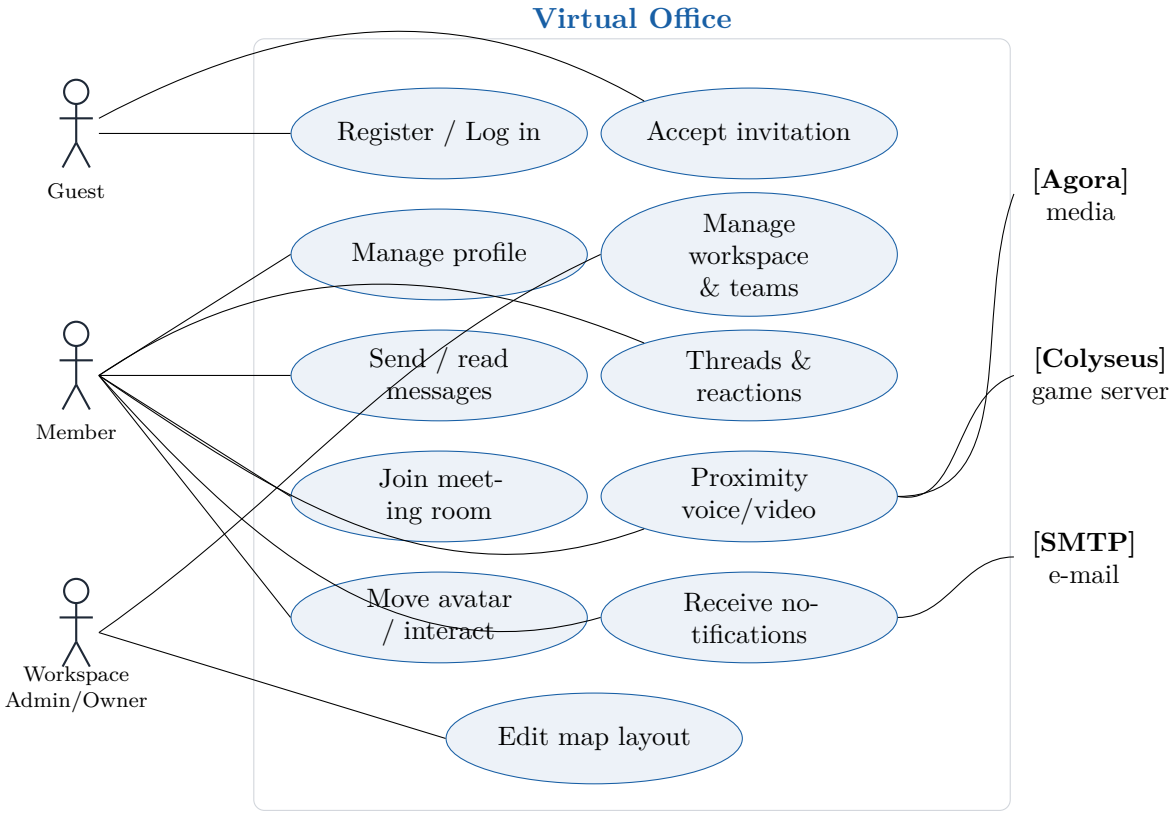


Figure 3.1. Top-level use-case diagram for VIRTUAL OFFICE.

3.4.1 Detailed Use Cases

Two representative use cases are specified in full to illustrate the level of interaction the system supports.

3.5 User Stories

The requirements can also be read as user stories, which capture intent from the user's perspective:

- *As a guest*, I want to register and be invited into a workspace so that I can join my team's virtual office.
- *As a member*, I want to see my colleagues moving around the office so that I know who is present and available.
- *As a member*, I want a conversation to start simply by walking up to a colleague so that I can ask a quick question without scheduling a meeting.
- *As a member*, I want persistent channels, threads, and direct messages so that important discussion is not lost when a call ends.
- *As a member*, I want to receive a notification on my phone when I am mentioned so that I do not miss time-sensitive messages.

Table 3.3. Use case UC-05: Send a message to a channel.

Field	Description
Use case	Send a message to a channel
Actor	Member
Preconditions	The member is authenticated and is a member of the target channel; a WebSocket connection is established.
Main flow	1. The member types a message and presses send. 2. The client generates a client message id and publishes the message over STOMP to `/app/chat/send`. 3. The chat service verifies channel membership, persists the message (idempotent on the client id), and broadcasts a `NEW_MESSAGE` event to `/topic/channel/{id}`. 4. All subscribed clients, including the sender, render the message.
Alternative flows	2a. If the socket send fails (mobile), the client falls back to a REST `POST` of the same message. 3a. If the member is not a member of the channel, the service returns a `NOT_A_MEMBER` error on the member's private error queue.
Postconditions	The message is stored and visible to all channel members; unread counts update for other members.

Table 3.4. Use case UC-08: Proximity voice/video in the 2D office.

Field	Description
Use case	Start a proximity call
Actor	Member (two members)
Preconditions	Both members are in the same office room, have enabled their webcam, and are marked ready to connect.
Main flow	1. Member A moves their avatar toward member B. 2. When the avatars overlap, the client detects the proximity via physics and, to avoid double-calling, the member with the greater session id initiates a PeerJS call. 3. The other member auto-answers; encrypted peer-to-peer audio/video flows directly between them. 4. Each sees the other's video tile appear.
Alternative flows	2a. If either member is not ready or has no webcam, no call is placed. 4a. If the two avatars are inside the designated private-meeting rectangle, the call persists even after they stop overlapping.
Postconditions	A live call exists until the members separate (outside the private area), at which point it is torn down.

- *As an admin*, I want to invite and organise members into teams and arrange the office map so that the space reflects how my organisation works.
- *As an owner*, I want confidence that only authorised users can join my workspace's rooms and read its channels.

3.6 Domain Model

Figure 3.2 shows the core domain entities and their relationships, abstracted across services. It is a conceptual model: in the implementation these entities are owned by different services and stored in different databases (see Chapters 4 and 5), but conceptually they form a single connected model centred on the *Workspace* and the *User*.

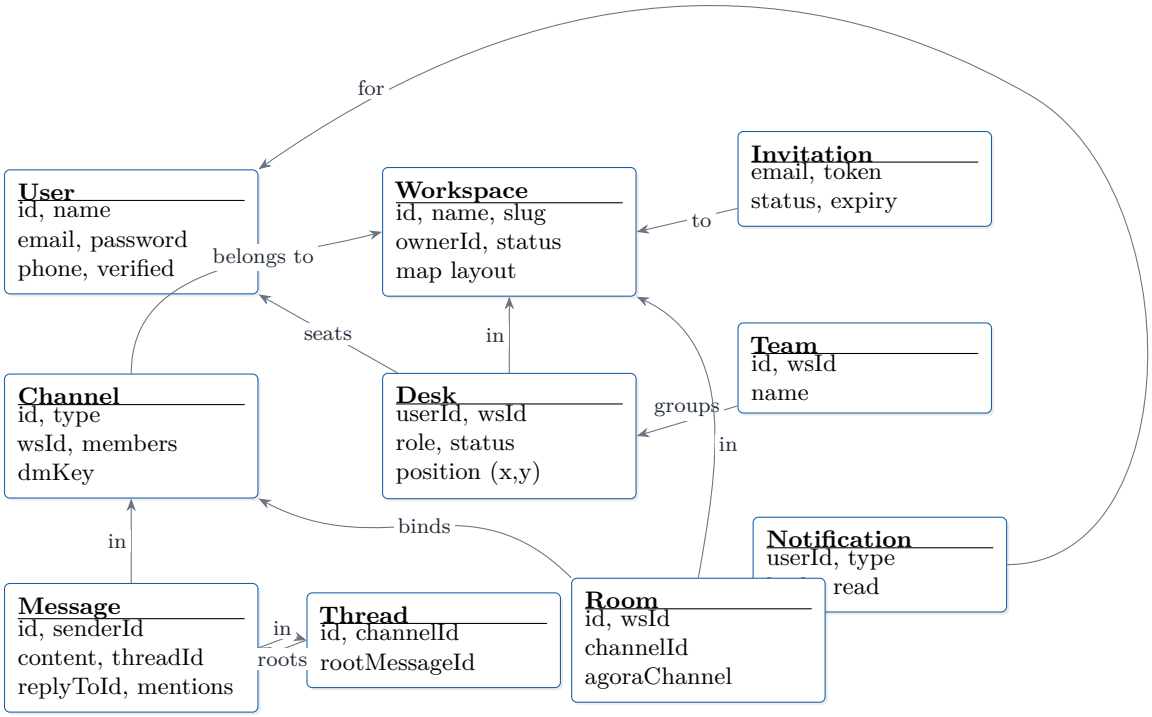


Figure 3.2. Conceptual domain model of VIRTUAL OFFICE (cross-service view).

The central relationship is that a *User* takes a *Desk* in a *Workspace*; the desk carries the user's workspace-scoped role, profile, and avatar position. Everything else — channels, rooms, teams, invitations — is scoped to the workspace, and messaging (messages and threads) hangs off channels. This model is realised in the following chapters, where each entity's ownership, storage technology, and API are described in full.

CHAPTER 4

System Architecture

This chapter presents the architecture of Virtual Office as a whole: the decomposition into microservices, the clients that consume them, the gateway that guards them, and the cross-cutting mechanisms — authentication, synchronous and asynchronous communication, real-time messaging, and persistence — that make the parts work as a coherent system. Chapter 5 then descends into each service in detail; here the goal is the big picture and the patterns that repeat everywhere.

4.1 Architectural Style

Virtual Office follows a **microservices** architecture with an **API gateway** at the edge and an **event-driven** backbone for cross-service communication. The guiding principles are:

One capability per service. Each service owns a single business capability — identity, workspaces, chat, rooms, notifications — and nothing else. A service is the sole owner of its data; no other service reaches into its database.

Security at the edge. Authentication is verified once, at the gateway. Internal services trust identity that the gateway has already validated and forwarded, and never re-implement token parsing.

Synchronous where an answer is needed, asynchronous where it is not. A service calls another synchronously only when it needs an immediate answer (for example, "what is this user's role in this workspace?"). For changes that other services should merely *react* to, it publishes an event and moves on.

Polyglot persistence. Each service uses the database that best fits its data: a relational store for structured, transactional data; a document store for flexible, high-volume data; and an in-memory store for ephemeral state and caches.

4.2 System Overview

Figure 4.1 shows the full topology: three families of clients, the gateway, the nine backend modules, the message broker, and the datastores.

Honest note on maturity. Of the nine backend modules, five are implemented services (user, workspace, chat, room, notifications), two are unimplemented Maven scaffolds (**calendar** and **tasks** — only a POM, properties, and README each), and one (**shared-library**) is a placeholder module with no code. This is stated plainly wherever these modules appear, so that the architecture is described as it is, not as intended.

4.3 Service Catalogue

Table 4.1 catalogues the backend modules with their port, responsibility, persistence, and messaging role. Two facts are worth flagging because they correct common assumptions:

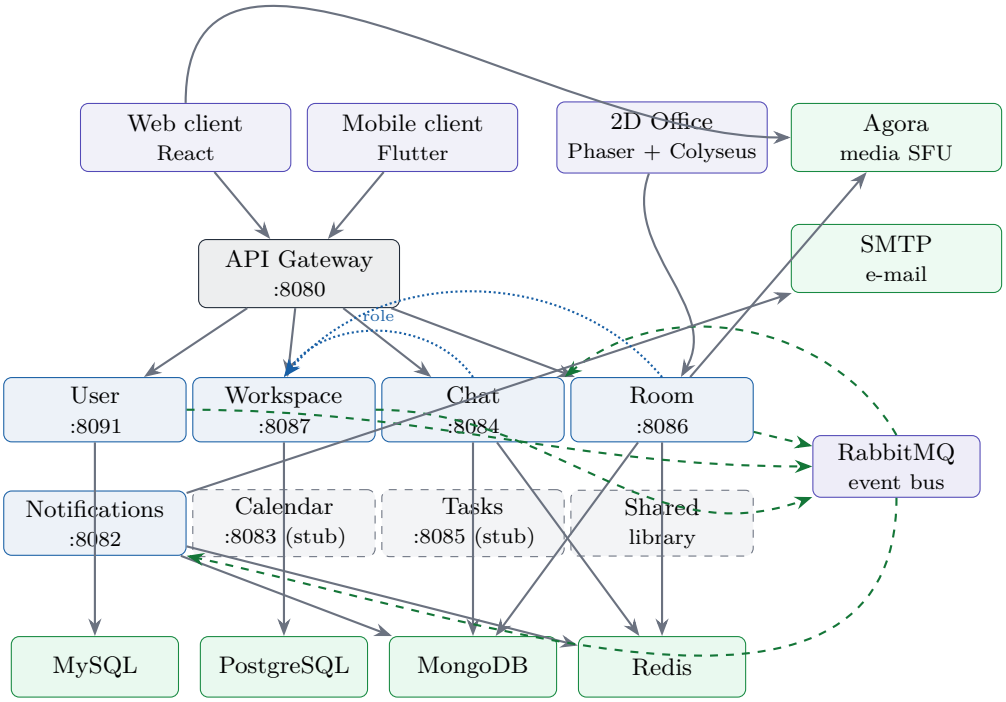


Figure 4.1. Virtual Office system architecture. Solid grey arrows: request/response and storage; dashed green: asynchronous events over RabbitMQ; dotted blue: synchronous internal role lookups; dashed boxes: unimplemented scaffolds.

the event bus is **RabbitMQ** (there is no Kafka anywhere in the system), and internal service-to-service HTTP uses Spring's `RestClient` (not Feign, despite an unused Feign dependency lingering on one classpath).

Table 4.1. Backend service catalogue.

Service	Port	Responsibility	Datastore	Messaging
Gateway API	8080	Edge routing, JWT validation, header injection	—	—
User	8091	Registration, login, JWT issuance, profile, OTP	MySQL	Produces `NotificationEvent`
Workspace	8087	Workspaces, desks, teams, invitations, map layout, roles	PostgreSQL + Flyway	Produces `WorkspaceChannelEvent`
Chat	8084	Channels, DMs, messages, threads, read receipts, real-time	MongoDB + Redis	Consumes room & workspace events
Room	8086	Voice/video rooms, presence, Agora tokens, proximity voice	MongoDB + Redis	Produces `RoomChannelEvent`
Notifications	8082	E-mails + in-app notifications, real-time push	MongoDB + Redis	Consumes `NotificationEvent`
Calendar	8083	*(scaffold)* scheduling / meetings	—	—
Tasks	8085	*(scaffold)* task assignment / tracking	—	—
Shared library	—	*(placeholder)* intended common DTOs/utilities	—	—

4.4 The API Gateway and Security Model

Every client request enters through the **gateway** (Spring Cloud Gateway, reactive). The gateway is the system's single security trust boundary, and its filter runs before routing with three responsibilities.

1. **Anti-spoofing.** It strips any client-supplied `X-User-Id` and `X-User-Role` headers from every incoming request. This is what makes it *safe* for internal services to trust those headers later — they can only ever be set by the gateway.
2. **Edge isolation of internal endpoints.** Any path under `/api/internal/**` is rejected with `403` at the edge, so server-to-server endpoints are unreachable from outside.
3. **Authentication.** Public paths (`/api/auth/*`, `/api/otp/*`, and any `*/health`) pass through untouched. Every other request must carry a valid `Bearer` JWT; the gateway verifies the HS256 signature and expiry, extracts the `userId` claim, and injects `X-User-Id` (and a default `X-User-Role`) before forwarding.

Figure 4.2 shows the end-to-end authentication flow, from credential exchange to an authenticated call reaching a downstream service.

Beyond the JWT, the system uses a second, coarser credential for **server-to-server** calls: internal endpoints (workspace role lookups, the Colyseus session/presence API) are authenticated with a shared `X-Internal-Token`. Because such endpoints are blocked at the gateway, this token only ever travels inside the trusted network. Together, the two mechanisms give a clean separation: *user* identity via JWT at the edge, and *service* identity via a shared token behind it.

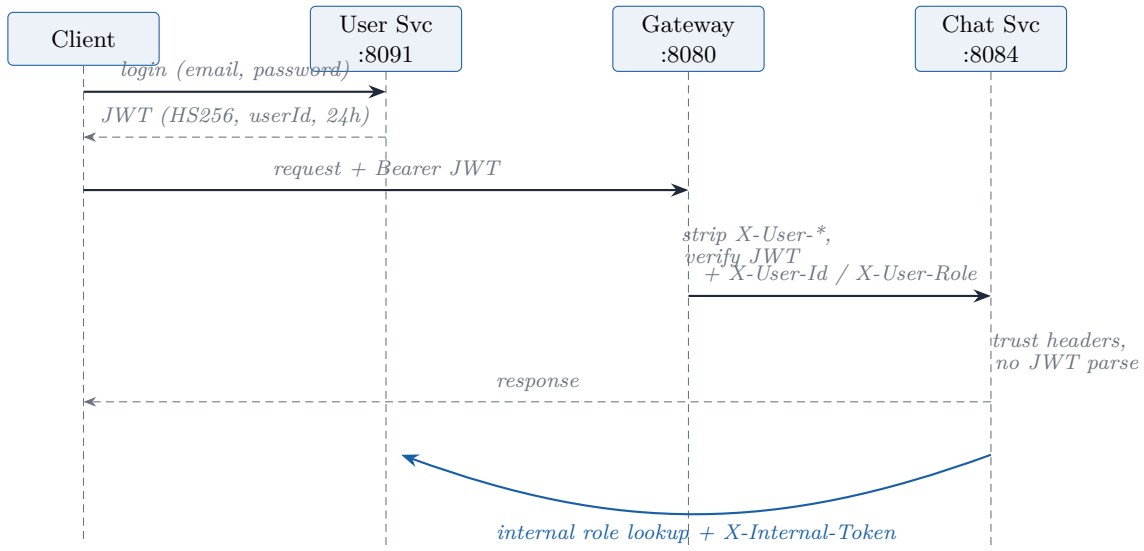


Figure 4.2. End-to-end authentication and identity-propagation flow.

4.5 Communication Patterns

4.5.1 Synchronous: internal REST

When a service needs an immediate answer from another, it makes a direct HTTP call using Spring's `RestClient`. In VIRTUAL OFFICE the dominant example is authorisation: both the chat and room services must know a caller's role *within a specific workspace* before allowing an action, so they call the workspace service's internal endpoint `/api/internal/workspace/{id}/members/{userId}/role`. These calls are designed to **fail closed**: if the workspace service is unreachable, the caller denies the action (returning `503`) rather than allowing it, so a dependency outage cannot become a security hole.

4.5.2 Asynchronous: the RabbitMQ event bus

Most cross-service communication is asynchronous and event-driven. A service that changes state publishes an *event* to a RabbitMQ `DirectExchange`; a consuming service, bound to a durable queue by a routing key, reacts to it. The producer neither knows nor waits for the consumer. Table 4.2 lists every event flow in the system.

Table 4.2. Event flows over RabbitMQ. All use durable direct exchanges, JSON payloads, and consumer-side retry with exponential backoff.

Producer	Exchange	Routing key	Consumer	Event / effect
User	`notifications.exchange`	`routing.key`	Notifications	`NotificationEvent`: send e-mail / in-app notice
Room	`room.exchange`	`room.channel.event`	Chat	`RoomChannelEvent`: create/sync a `ROOM` channel
Workspace	`workspace.exchange`	`workspace.channel.event`	Chat	`WorkspaceChannelEvent`: maintain the canonical workspace channel

This design has three payoffs that recur through the book. First, **decoupling**: the room service can create a room and return immediately, while the chat service materialises the bound channel a moment later. Second, **resilience**: if the chat service is momentarily down,

its events wait durably in the queue and are processed on recovery. Third, **idempotency**: every event carries a unique `eventId`, and consumers check it before acting, so a redelivered message is harmless. The price is **eventual consistency** — a freshly created room's channel may `404` for a brief window until its event is processed — which clients are designed to tolerate ("setting up chat…").

4.6 The Real-time Layer

Three services — chat, room, and notifications — expose real-time functionality over STOMP-on-WebSocket. They share two patterns so consistently that it is worth describing them once, centrally.

4.6.1 Ticket-based WebSocket authentication

Because a browser cannot set an `Authorization` header on a WebSocket handshake, each real-time service authenticates the socket with a single-use *ticket*, illustrated in Figure 4.3:

1. The client makes an authenticated REST call, `POST .../ws-ticket`. The service generates a random ticket and stores it in Redis, mapped to the caller's identity, with a 60-second time-to-live.
2. The client opens the WebSocket with the ticket as a query parameter, e.g. `/api/chat/connect?ticket=`
3. A handshake interceptor atomically reads-and-deletes the ticket from Redis (`GETDEL`), binding the resulting identity to the STOMP session. The ticket cannot be reused.

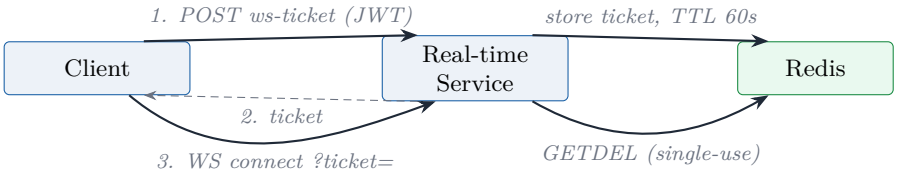


Figure 4.3. Ticket-based WebSocket handshake, used identically by the chat, room, and notification services.

4.6.2 The `{action, payload}` envelope

Every real-time message the server pushes uses the same envelope: a JSON object with an `action` string and a `payload`. A chat client, for instance, receives `{action:"NEW_MESSAGE", payload:{...}}`, `{action:"EDIT_MESSAGE", ...}`, or `{action:"TYPING", ...}`; a room client receives `PARTICIPANT_JOINED`, `STATE_CHANGED`, or `ROOM_CLOSED`; a notification client receives `NEW_NOTIFICATION`. This uniformity means every client can use one dispatch loop — switch on `action`, handle the `payload` — regardless of service, which noticeably simplifies the web and mobile code (Chapters 7 and 8).

STOMP *destinations* follow an equally regular scheme: public broadcasts go to `/topic/...` (a channel, thread, or room), a user's private stream goes to `/user/queue/...` (errors, notifications), and clients send to `/app/...` application destinations. Subscriptions to sensitive topics are authorised by an interceptor that checks membership before allowing the `SUBSCRIBE`.

4.7 Polyglot Persistence

Each service stores its data in the technology that fits it best (Table 4.1):

MySQL — User service. Accounts and verification requests are structured, relational, and transactional; a relational database with JPA is the natural home.

PostgreSQL with Flyway — Workspace service. The workspace domain is the most structurally rich in the system (workspaces, desks, teams, zones, map layers, tilesets, invitations) with many foreign-key relationships and optimistic-locking needs; PostgreSQL provides that, and Flyway migrations give the schema first-class version control.

MongoDB — Chat, Room, Notifications. Messages, rooms, and notifications are high-volume, append-heavy documents whose shape varies (mentions, attachments, arbitrary notification `data`); a document store fits, and each service uses its own database on the shared instance.

Redis — Chat, Room, Notifications. Ephemeral and latency-critical state lives in Redis: WebSocket tickets, read-receipt cursors, room presence and heartbeats, rate-limit counters, and idempotency claims. Keys are namespaced per service (`read:*`, `room:*`, `notif:*`) to share one instance safely.

4.8 Technology Stack Summary

Table 4.3 consolidates the technologies used across the platform, by tier. The clients are covered in Chapters 6–8.

Table 4.3. Technology stack by tier.

Tier	Technologies
Backend services	Java 21, Spring Boot (3.4.1–4.0.x), Spring Cloud Gateway, Spring Web/WebFlux, Spring Data JPA/Mongo/Redis, Spring AMQP, Spring WebSocket + STOMP, jjwt, MapStruct, Flyway
Data & messaging	MySQL, PostgreSQL, MongoDB, Redis, RabbitMQ
Web client	React 18, MUI 7, Tailwind CSS, `@stomp/stompjs`, `agora-rtc-sdk-ng`, axios, React Router 7
Mobile client	Flutter / Dart, Provider, `http`, `stomp_dart_client`, `awesome_notifications`, `shared_preferences`
2D office	TypeScript, Phaser 3, Colyseus, PeerJS/WebRTC, React + Redux Toolkit, Vite
External services	Agora (media SFU), Gmail SMTP (e-mail)
Build & ops	Maven (multi-module), Docker Compose, health endpoints

4.9 Deployment View

For development, each service runs on its documented port on `localhost`, and its infrastructure dependencies (MySQL, PostgreSQL, MongoDB, Redis, RabbitMQ) are brought up through Docker Compose definitions shipped with the services. Spring Boot's Docker Compose integration lets a service start its own dependencies automatically. In a containerised deployment, the gateway is the only publicly exposed service; all others sit on an internal

network, reachable only through the gateway or, for internal endpoints, only by peers holding the shared internal token. Service URLs are externalised as environment variables (for example `CHAT_SERVICE_URL`), so the same images run unchanged from a laptop to a cluster.

The chapters that follow now open each of these boxes in turn: the backend services (Chapter 5), the 2D office (Chapter 6), and the web and mobile clients (Chapters 7 and 8).

CHAPTER 5

Backend Implementation

This chapter documents each backend service in depth. For every implemented service it describes the responsibility, technology stack, data model, REST and WebSocket API, event interactions, and the notable business logic that gives the service its character. The two unimplemented scaffolds and the placeholder shared library are described honestly for completeness. Throughout, the cross-cutting patterns of Chapter 4 — gateway-injected identity, ticket-based sockets, the `{action, payload}` envelope, and idempotent event handling — appear in their concrete form.

5.1 Gateway API

The gateway (`gateway-api`, port 8080) is a reactive Spring Cloud Gateway application with no datastore. Its routing table maps public path prefixes to downstream services (Table 5.1), each target overridable by an environment variable for containerised deployment.

Table 5.1. Gateway routing rules.

Route	Path predicate	Target (default)
user-service	`/api/auth/**`, `/api/otp/**`, `/api/users/**`	`localhost:8091`
workspace-service	`/api/workspace/**`	`localhost:8087`
chat-service	`/api/chat/**`	`localhost:8084`
room-service	`/api/rooms/**`	`localhost:8086`

The security core is a single global filter, `AuthGatewayFilter`, ordered ahead of routing. Its logic captures the trust model of the whole platform:

```
// order -1: runs before routing
public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain
    chain) {
    ServerHttpRequest req = exchange.getRequest();
    String path = req.getURI().getPath();

    // 1. anti-spoofing: never let a client set identity headers
    ServerHttpRequest cleaned = req.mutate()
        .headers(h -> { h.remove("X-User-Id"); h.remove("X-User-Role");
            })
        .build();

    // 2. internal endpoints are unreachable from the edge
    if (path.startsWith("/api/internal/"))
        return forbidden(exchange);            // 403

    // 3. public passthrough
    if (path.endsWith("/health") || isPublicPrefix(path))
        return chain.filter(exchange.mutate().request(cleaned).build());

    // 4. authenticate: require + verify Bearer JWT, inject trusted
        identity
```

```
20      return jwtService.verifyAndExtractUserId(bearer(req))
21          .map(userId -> cleaned.mutate()
22              .header("X-User-Id", userId.toString())
23              .header("X-User-Role", "USER")
24              .build())
25          .map(r -> chain.filter(exchange.mutate().request(r).build()))
26          .orElse(unauthorized(exchange));        // 401
27  }
```

Listing 5.1. Representative gateway security filter (the platform's trust boundary).

The JWT itself is verified with HS256 using a shared secret (`jwt.secret`) identical to the one the user service signs with; the filter reads only the `userId` claim. A `CorsWebFilter` runs even earlier so that preflight `OPTIONS` requests never reach the auth filter. Because the gateway hard-codes the injected role to `USER`, platform-level RBAC is not yet modelled; all role decisions happen per-workspace in the workspace service (Section 5.3).

5.2 User Service

Responsibility. The user service (port 8091) is the identity authority: registration, login and JWT issuance, profile and profile-picture management, password change, and OTP-based verification. It is the origin of every JWT in the system and the producer of user-lifecycle notification events.

Stack. Spring Boot with Web, Data JPA, Security, and AMQP; a MySQL datastore; `jjwt` for tokens; and BCrypt for password and OTP hashing. It has no WebSocket surface and sends no e-mail itself — that is delegated to the notification service via events.

Data model. Two JPA entities (Table 5.2). A `User` owns many `VerificationRequests`; the profile picture is stored inline as a `LONGBLOB` with its MIME type.

Table 5.2. User service entities.

Entity (table)	Fields
`User` (`users`)	`id`, `firstName`, `lastName`, `email` (unique), `phoneNumber`, `password` (BCrypt), `accountStatus` (`ACTIVE`/`INACTIVE`/`PENDING_REPORT_REVIEW`/`SUSPENDED`), `isEmailVerified`, `isPhoneVerified`, `isDisabled`, `profilePicture` (`byte[]`), `profilePictureType`
`VerificationRequest` (`verification_requests`)	`id`, `otp` (BCrypt hash), `status` (`PENDING`/`APPROVED`), `createdAt`, `expiresAt` (+10 min), `type` (`PASSWORD_RESET`/`EMAIL_VERIFICATION`/`CHANGE_EMAIL`), `user` (`@ManyToOne`)

API. The REST surface (Table 5.3) splits into public authentication endpoints and protected profile endpoints; the latter are guarded by the service's own `JwtAuthFilter` for defence in depth, in addition to the gateway.

Authentication internals. On login, after a BCrypt password check, `JwtUtil` mints an HS256 token whose subject is the e-mail, with a custom `userId` claim and a 24-hour expiry. This is the token the gateway later verifies. Security is stateless (no server session); a `CustomUserDetailsService` loads users by e-mail and rejects disabled accounts, mapping a `SUSPENDED` status onto a locked account. A `GlobalExceptionHandler` maps authentication failures to `401`, integrity violations (duplicate e-mail) to `409`, and validation errors to `400`.

Table 5.3. User service REST API.

Method	Path	Purpose
POST	`/api/auth/register`	Register, hash password, issue JWT, emit `SIGNUP_SUCCESS`; `409` if e-mail exists
POST	`/api/auth/login`	Authenticate and issue JWT; `401` on failure
POST	`/api/auth/otp/request`	Generate and e-mail an OTP; emit `OTP`
POST	`/api/auth/otp/verify`	Verify an OTP, mark `APPROVED`
PUT	`/api/users/me/password`	Change password (checks old); emit `PASSWORD_RESET_SUCCESS`
GET	`/api/users/me`	Current profile
POST/GET	`/api/users/me/photo`	Upload / return profile picture (≤10 MB)
GET	`/api/users`	List users (id, name, e-mail)

Events produced. On the three lifecycle actions, the service publishes a `NotificationEvent` to the `notifications.exchange`:

```
{ "eventId": "9f1c...-uuid", "type": "OTP", "occurredAt":
    "2026-07-10T12:00:00Z",
  "payload": { "email": "user@x.com", "firstName": "Sara",
               "otp": "418306", "expiresInMinutes": 10 } }
```

Listing 5.2. A `NotificationEvent` (OTP request).

Housekeeping. A scheduled task runs hourly to bulk-delete expired verification requests, keeping the table small. Configuration points MySQL at `virtual_office`, uses JPA `ddl-auto=update` (a convenience with a known schema-drift risk in production), and connects to RabbitMQ on the default port.

5.3 Workspace Service

Responsibility. The workspace service (port 8087) is the spatial and organisational backbone of the platform. It owns *workspaces* (companies), *desks* (a user's seat, profile, position, and workspace role), *teams*, *invitations*, and the editable *2D map layout* (tilesets, layers, zones, spawn points, map objects). It is also the authority on *workspace roles*, and it exposes an internal API that boots and synchronises the Colyseus game sessions.

Stack. The most conventionally "enterprise" service: Spring Web, Data JPA, Validation, AOP, and AMQP over **PostgreSQL**, with **Flyway** owning the schema (`ddl-auto=validate`), **MapStruct** for DTO mapping, and **springdoc-openapi** publishing a Swagger UI. Its build enforces 90% test coverage via JaCoCo.

Data model. The richest schema in the system (Table 5.4); Figure 5.1 sketches the central relationships. The `Workspace` carries an optimistic-locking `layoutVersion`; the `Desk` is the join between a user and a workspace and holds the workspace-scoped `role`.

API highlights. Beyond CRUD on workspaces, teams, desks, and map objects, three areas stand out. *Invitations*: an admin invites by e-mail (creating a `PENDING` invitation and desk and an e-mail event); the invitee accepts via a tokenised link, which creates or reactivates their desk and fires a "member added" event to the chat service. *Layout editing*: replacing the map is guarded by optimistic concurrency — the client submits the version it edited, and a mismatch is rejected — after which child rows (layers, tilesets, zones)

Table 5.4. Principal workspace-service entities (PostgreSQL).

Entity	Key fields
`Workspace`	`id`, `name`, `slug` (unique), `ownerId`, `status`, `visibility` (`INVITE_ONLY`), `inviteToken`, `tileSize`, `mapWidth/Height`, `mapGeometry` (jsonb), `layoutVersion` (`@Version`)
`Desk`	`userId`, `workspaceId`, `fullName`, `avatarCharacter`, `status`, `statusEmoji`, `positionX/Y`, `isOnline`, `lastSeenAt`, `role` (`GUEST<MEMBER<ADMIN<OWNER`), `teamId`, `inviteStatus`, `isActive` — unique on `(userId, workspaceId)`
`Team`	`id`, `workspaceId`, `name` — unique on `(workspaceId, name)`
`WorkspaceInvitation`	`token` (UUID), `email`, `status`, 7-day expiry — one `PENDING` per `(workspaceId, email)`
`Zone`	`type`, `x`, `y`, `width`, `height`, `voiceRoomId` (unique), `proximityRadius` — read by the room service
`MapObject`	`type` (`COMPUTER`/`WHITEBOARD`), `positionX/Y`, `roomId` (stable UUID), `capacity`, `isActive`
`MapLayer`, `Tileset`, `SpawnPoint`	The Tiled map: gid layer matrices (jsonb), tilesets (unique `firstGid`), spawn points

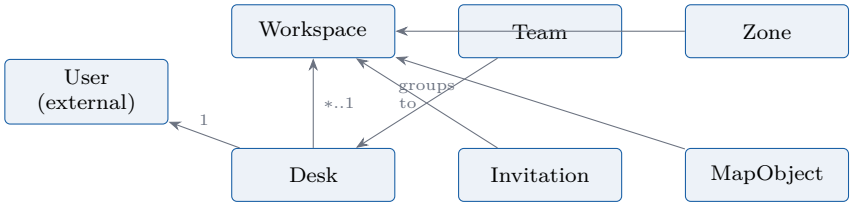


Figure 5.1. Workspace-service core relationships. A Desk joins a User to a Workspace and carries the workspace role.

are validated and replaced atomically. *Internal session API*: under `/api/internal/**` (gateway-blocked, `X-Internal-Token`-guarded), the service serves the Colyseus game server a full `/session-config` boot payload, validates joins, resolves `/members/{userId}/role` (consumed by the chat and room services), and ingests avatar `/presence` updates.

Events produced. On workspace creation and membership change, a `WorkspaceChannelEvent` is published — *after the database transaction commits* — to the `workspace.exchange`, so the chat service can maintain a single canonical "general" channel per workspace. Publishing after-commit ensures the chat service never reacts to a change that was later rolled back, and a broker outage is logged rather than allowed to fail the user's request.

5.4 Chat Service

Responsibility. The chat service (port 8084) is the real-time messaging core: channels (group, direct, and room-bound), messages (with edit, soft-delete, reply, mention, and idempotent send), threads, and read receipts/unread counts. It delivers over WebSocket and provisions channels in response to events from the room and workspace services.

Stack. Spring Web, WebSocket, Data Mongo, Data Redis, AMQP, and Validation over **MongoDB** (documents) and **Redis** (cursors, tickets, rate limits). It consumes events but produces none.

Data model. Three collections (Table 5.5). A subtle but important distinction is `threadId` ("this message lives inside a thread") versus `replyToId` ("this message quote-replies another") — the two are independent. Message identity uses MongoDB `ObjectId`s, while channels and threads use integer ids from a sequence-counter collection.

Table 5.5. Chat service collections.

Collection	Fields
`channels`	`id`, `name`, `type` (`GROUP`/`DIRECT`/`ROOM`), `workspaceId`, `members` (`int[]`), `dmKey` ("`minId_maxId`"), `canonical`, `createdBy`, timestamps
`messages`	`id` (ObjectId), `channelId`, `senderId`, `senderRole` (captured at send), `content`, `type` (`TEXT`/`SYSTEM`), `threadId`, `replyToId`, `mentions`, `clientMessageId` (idempotency), `deleted`, timestamps
`threads`	`id`, `channelId`, `rootMessageId` (unique), `name`, `createdBy`, `creatorRole`, `deleted`, timestamps

Carefully chosen indexes make the store correct and fast: a unique `(senderId, clientMessageId)` index enforces idempotent sends; a unique, sparse `dmKey` index enforces exactly one direct-message channel per user pair; and a *partial* unique `(workspaceId, name)` index that applies only to `GROUP` channels lets room channels reuse names freely.

Authorisation. Every channel- or thread-scoped operation checks membership (`403` otherwise). Editing is restricted to the author. Deletion follows a nuanced rule that survives later role changes because the author's role is *captured on the message*: the owner may always delete; an `ADMIN` may delete others' messages *except* those authored by another `ADMIN`. Before creating a channel or room-bound resource, the service confirms the caller's workspace role by calling the workspace service's internal role endpoint, failing closed if it is unreachable.

Read receipts. Rather than store a read row per message, the service keeps a single forward-only cursor per user per channel in Redis (`read:{channelId}:{userId}`), advanced with a Lua script and expiring after thirty days; the unread count is then simply the number of messages whose id exceeds the cursor.

Real-time API. The WebSocket endpoint is `/api/chat/connect`, authenticated by the ticket pattern of Section 4.6. A subscription interceptor authorises each `SUBSCRIBE` to `/topic/channel/{id}` or `/topic/thread/{id}` against membership, returning a typed error (`NOT_A_MEMBER`, `CHANNEL_NOT_FOUND`, `INVALID_PAYLOAD`) on the user's private `/user/queue/errors` stream. Clients `SEND` to `/app/chat/send` and `/app/chat/typing`; the server persists and broadcasts, using the uniform envelope with actions `NEW_MESSAGE`, `EDIT_MESSAGE`, `DELETE_MESSAGE`, `THREAD_DELETED`, and `TYPING`. Figure 5.2 traces a send.

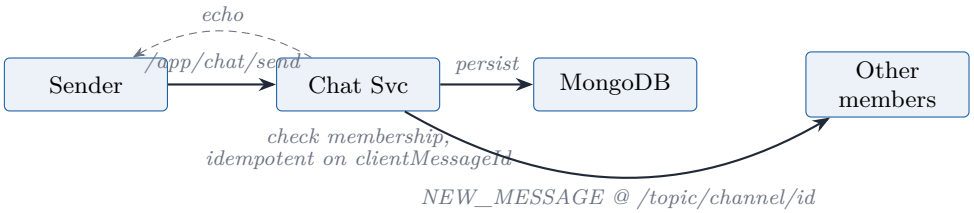


Figure 5.2. Message-send flow in the chat service.

Events consumed. Two listeners keep the channel space in sync with the rest of the platform. A `RoomChannelEvent` (from the room service) creates, deletes, or edits the membership of a `ROOM`-type channel; a `WorkspaceChannelEvent` (from the workspace service) maintains the single canonical group channel of each workspace. Both are idempotent on `eventId` and retried with backoff, so redelivery is safe and a transient failure self-heals.

5.5 Room Service

Responsibility. The room service (port 8086) coordinates voice/video rooms and the proximity-voice layer of the 2D office. It owns room identity and membership (in MongoDB) and live presence (in Redis), mints Agora media tokens, pushes presence over WebSocket, computes proximity voice groups from avatar positions, and publishes room-channel events to the chat service. It is explicitly a *coordinator*, not a media server — audio and video never touch it.

Data model. A `Room` document binds three identities: its own `ObjectId`, the `channelId` of the chat channel it is bound to (generated by the room service and published to chat), and a deterministic `agoraChannelName` (`room-<hex>`). Live presence is entirely ephemeral in Redis: a participants hash per room (each participant's mute, camera, and screen-share state), a short-lived "alive" key per participant refreshed by heartbeats, and the WebSocket ticket. A capacity limit (default 25) is enforced *atomically at join* by a Lua script that adds the participant only if the room is not full.

```
-- KEYS[1] = room:{id}:participants   ARGV[1] = userId   ARGV[2] = max
if redis.call('HLEN', KEYS[1]) >= tonumber(ARGV[2]) then
    return 0                                    -- ROOM_FULL
end
redis.call('HSET', KEYS[1], ARGV[1], ARGV[3])  -- ARGV[3] = state json
return 1
```

Listing 5.3. Representative atomic capacity check at join (Redis Lua).

API. Rooms are created, listed, updated, and deleted under `/api/rooms`; members are added and removed; and the central action is `POST /api/rooms/{id}/join`, which re-checks workspace membership, registers presence, **mints an Agora token**, broadcasts `PARTICIPANT_JOINED`, and returns the room, the Agora channel name, the token, and the current participant list. Over WebSocket (`/ws/rooms`), clients send `/app/room/state` (mute/camera/screen changes, rebroadcast as `STATE_CHANGED`) and `/app/room/heartbeat` (to refresh their alive key), and subscribe to `/topic/room/{roomId}` for `PARTICIPANT_JOINED/LEFT`, `ROOM_UPDATED`, and `ROOM_CLOSED`.

Agora token minting. The service builds an Agora AccessToken v2 by hand: it packs the app id, channel, uid, privileges (join, publish audio, publish video, data) and expiry, signs the message with HMAC-SHA256 using the app certificate, and zlib-compresses the result into the familiar "`007...`" token. The media then flows client-to-Agora directly; the backend never sees it.

Proximity-voice engine. The most novel piece of the service turns avatar positions into voice groups deterministically. On each position snapshot from the game server (pushed to the internal `/positions` endpoint), the engine assigns any avatar standing inside a zone that has a `voiceRoomId` to that zone's private channel, and clusters the remaining open-area avatars by a union-find over mutual hearing radius; a cluster of two or more becomes an ad-hoc channel named `prox-<workspaceId>-<smallestUserId>`, while a lone avatar gets no channel. Only avatars whose group changed are notified, via a `VOICE_GROUP_CHANGED` broadcast, and the endpoint returns the mapping of each user to their Agora channel. This is the *signalling* layer that decides *who should hear whom*; Agora carries the actual audio.

Events produced. On room create/delete and membership change, a `RoomChannelEvent` is published to the `room.exchange`; the chat service consumes it to materialise or update the bound `ROOM` channel. Room creation even *rolls back* if the event cannot be published, so a room never exists without its channel eventually appearing.

5.6 Notification Service

Responsibility. The notification service (port 8082) has two jobs: sending transactional **e-mails** (sign-up, login, OTP, password reset) from versioned HTML templates, and delivering **in-app notifications** (currently `TASK_ASSIGNED`) by persisting them and pushing them over WebSocket. It is a pure consumer of events and never publishes.

Stack. Spring Web, Data Mongo, Data Redis, AMQP, Mail, and WebSocket. An in-app `Notification` document records `userId`, `type`, `title`, `body`, an arbitrary `data` map, `read` state, and a unique `eventId` for idempotency; e-mail events are not persisted.

Delivery. A single `@RabbitListener` on `notifications.queue` dispatches on the event type. E-mail events take an idempotency claim in Redis (`SETNX notif:processed:{eventId}`, 24-hour TTL), render the matching HTML template by `{{placeholder}}` substitution, and send via SMTP — releasing the claim on failure so a retry can re-attempt. The `TASK_ASSIGNED` event instead builds a notification, saves it to MongoDB (the unique `eventId` making the insert idempotent), and pushes a `{action:"NEW_NOTIFICATION", ...}` envelope to the recipient's private queue via Spring's user-destination routing. Consumption retries five times with exponential backoff and then drops the message (there is no dead-letter queue in this version).

Inbox API. The web client reads history through `GET /api/notifications` (paged), an `/unread-count`, and mark-read and delete operations, plus the usual `/ws-ticket` for the socket. Because offline users cannot receive a push, the persisted history backfills them on their next inbox load — push is best-effort, storage is the source of truth.

5.7 The Unimplemented Modules

For an accurate account, three modules must be described as the placeholders they are.

Calendar service (port 8083). A Maven scaffold containing only a POM, a minimal `application.properties`, and a README describing intended features (event scheduling, meeting invites, calendar sync). It has no Java source and is not wired into the gateway.

Tasks service (port 8085). Likewise a scaffold. It is the intended future *producer* of the `TASK_ASSIGNED` events that the notification service already knows how to consume — so the consumer side of task notifications exists and waits for this producer.

Shared library. Declared as a module and intended to hold common DTOs, utilities, and exception handling, but currently containing no Java classes. Each service therefore defines its own exception handler, user-context helper, and DTOs independently.

These gaps are the clearest signposts for future work (Chapter 10).

5.8 Cross-service Consistency in Practice

It is worth closing the chapter by watching the patterns of Chapter 4 act together in a single scenario: *a user accepts a workspace invitation.*

1. The invitee opens the tokenised accept link; the request reaches the workspace service through the gateway with a validated `X-User-Id`.
2. The workspace service marks the invitation `ACCEPTED` and creates (or reactivates) the invitee's `Desk` in one transaction.
3. *After* that transaction commits, it publishes a `WorkspaceChannelEvent` (member added) to RabbitMQ and returns success — the user's request does not wait for anything downstream.
4. The chat service, subscribed to that exchange, consumes the event (idempotently on `eventId`) and adds the user to the workspace's canonical channel, so their new channel appears a moment later.

No service reached into another's database; identity was validated once; the cross-service change was asynchronous, durable, and idempotent; and the user experienced an immediate response with the rest catching up behind it. That combination — edge-validated identity, per-service data ownership, and event-driven eventual consistency — is the through-line of the entire backend.

CHAPTER 6

The 2D Virtual Office

The 2D office is what makes Virtual Office a *place* rather than a set of tools. This chapter describes it in full: its client/server split, the shared type contract that keeps the two in step, the multiplayer synchronisation model that keeps many avatars moving smoothly, the interactive objects that populate the world, and the peer-to-peer stack that carries proximity voice, video, and screen sharing. The component is an adapted fork of the open-source *SkyOffice* project, extended and integrated as one of Virtual Office's three clients.

6.1 Overview and Structure

The 2D office is a small but complete *multiplayer game*. It is organised as a monorepo of three workspaces (Table 6.1): a browser *client* that renders the world and the user interface, an authoritative *server* that holds the shared state, and a *types* package of shared definitions imported by both. Its one-line technology summary is: Phaser for the game, React and Redux for the interface, Colyseus for multiplayer state, and PeerJS/WebRTC for media.

Table 6.1. Structure of the 2D office component.

Workspace	Role	Stack
`client/`	Renders the world and UI; captures input; runs media	TypeScript, Phaser 3, React 18, Redux Toolkit, PeerJS, Vite
`server/`	Authoritative multiplayer state and message handling	TypeScript, Colyseus, Express, bcrypt
`types/`	Shared enums, message codes, and state interfaces	TypeScript

6.2 The Shared Type Contract

Because client and server are separate programs that must agree exactly on the shape of the world and the messages they exchange, the `types/` package is the linchpin: both sides import it, so a change to a message code or a state field is enforced by the compiler across the whole system. Its central definitions are:

- `IOfficeState` — the synchronised room state: a map of `players`, a map of `computers`, a map of `whiteboards`, and an array of `chatMessages`.
- `IPlayer` — `{name, x, y, anim, readyToConnect, videoConnected}`: everything replicated about an avatar.
- `IComputer` / `IWhiteboard` — each holds a set of the session ids currently connected to it.
- The `Message` enum — the remote-procedure opcodes exchanged over the socket (`UPDATE_PLAYER`, `CONNECT_TO_COMPUTER`, `ADD_CHAT_MESSAGE`, and so on).

- Small enums for `ItemType` (`CHAIR`, `COMPUTER`, `WHITEBOARD`, `VENDINGMACHINE`), `PlayerBehavior` (`IDLE`, `SITTING`), `RoomType` (`LOBBY`, `PUBLIC`, `CUSTOM`), and day/night background mode.

6.3 The Authoritative Server

The server is a Colyseus application. Its entry point creates an Express-backed Colyseus server and registers three room definitions: the built-in `LobbyRoom` (which tracks the live list of rooms), a persistent `PUBLIC` office room that is never disposed even when empty, and a `CUSTOM` room type for password-protectable private offices whose creation is broadcast to lobby clients in real time. A Colyseus monitor dashboard is mounted for live inspection.

Each office is a `SkyOffice` room extending `Room<OfficeState>`. Its lifecycle hooks encode the rules of the world:

`onCreate` optionally bcrypt-hashes a room password, instantiates the state, and populates it with a fixed set of five computers and three whiteboards (keyed `"0"`…`"4"` and `"0"`…`"2"`), then registers all message handlers.

`onAuth` for a protected room, compares the supplied password against the stored hash, rejecting a mismatch with a `403`.

`onJoin` inserts a new `Player` keyed by the connection's session id and sends that client the room's metadata.

`onLeave` removes the player and scrubs its session id from every computer's and whiteboard's connected-user set, so occupancy stays correct.

State is mutated through the **command pattern** (`@colyseus/command`): a dispatcher runs small command objects — `PlayerUpdateCommand` (write `x`, `y`, `anim`), the computer/whiteboard add-user and remove-user commands, and `ChatMessageUpdateCommand`, which appends a chat message and caps the history at the last 100 so late-joiners see recent context. A few trivial flag flips (marking a player ready, or relaying a "stop screen share" signal) bypass the dispatcher for directness.

6.4 The Client

The client runs two parallel worlds glued together by a shared Redux store and a Phaser event emitter: the **Phaser** game canvas (the animated office) and a **React** DOM overlay (dialogs, chat, buttons). Neither engine imports the other's render tree; they communicate only through the store and the event bus.

6.4.1 Scenes

Phaser organises a game into scenes. The office uses three:

Bootstrap preloads every asset — the Tiled tile-map, the tilesets, the item spritesheets, and the four character spritesheets (*Adam*, *Ash*, *Lucy*, *Nancy*) — constructs the network service (opening the WebSocket), and pre-checks media permissions.

Background draws a decorative parallax backdrop with a sun/moon and drifting clouds, switching palette between a day and a night theme.

Game is the core scene. It builds the character animations and the tile-map with its collision layer, spawns the local avatar and a *player-selector* probe, and reads the Tiled object layers to instantiate chairs, computers, whiteboards, and vending machines as physics bodies. It then registers colliders (avatar versus walls and objects) and two

crucial *overlap* handlers — the selector versus interactive items (which drive the "Press E/R" prompts) and the local avatar versus other avatars (which drives proximity calls) — and wires up the network callbacks.

6.4.2 Characters

Every avatar is a Phaser sprite with a companion container holding a floating name label and a transient speech bubble; its render depth is derived from its `y`-coordinate so that avatars correctly overlap in the 2.5D view. Three character classes specialise this base:

`MyPlayer` is the locally controlled avatar. Each frame it reads the keyboard (WASD or arrows) and the on-screen joystick, moves at a normalised 200 pixels per second (so diagonals are not faster), plays the matching walk or idle animation, and — crucially — sends an `UPDATE_PLAYER` message whenever its position or animation changes. Pressing E near a chair snaps it into a seated pose and state; pressing R opens a computer, a whiteboard, or a vending-machine link depending on what the selector has highlighted.

`OtherPlayer` is a remote avatar. It does not receive a continuous stream of positions; instead it stores a *target* position and interpolates toward it each frame at the same 200 px/s, snapping if the gap is large or the tab was backgrounded. It also owns the proximity-call trigger.

`PlayerSelector` is an invisible probe kept one tile ahead of the local avatar in its facing direction; whatever interactive item it overlaps becomes the current selection that E/R act upon.

6.4.3 The React and Redux overlay

The React layer is a small state machine driven by Redux: with no room joined it shows the room-selection dialog; with a room but no login it shows the name-and-avatar picker; once in the world it renders the chat panel, a prompt to enable the webcam, and the mobile joystick, overlaying a computer or whiteboard dialog when the user is at one of those objects. Five Redux slices hold the shared UI state — the user/session, the room list, the screen-share session, the whiteboard URLs, and the chat log — and it is through these slices that the Phaser world and the React interface stay in agreement.

6.5 The Multiplayer Synchronisation Model

The heart of the component is how many moving avatars are kept in sync efficiently. Colyseus is *authoritative*: it holds the one true `OfficeState`, every client holds a replica, and Colyseus transmits only the *changed fields* as compact binary diffs — moving one avatar never re-sends the whole world. On top of that, the client uses two classic game-networking techniques (Figure 6.1):

Client-side prediction. The local avatar moves *immediately* in response to input, without waiting for the server. Its new position is then sent to the server, so the player never feels input lag.

Remote interpolation. Other avatars are not snapped to each incoming position; each is eased smoothly from its current position toward the latest target, so sparse network updates still produce fluid motion.

The full loop is: `MyPlayer` moves locally and sends `UPDATE_PLAYER`; the server's `PlayerUpdateCommand` writes the change into that player's schema; Colyseus diffs it to every *other* client, whose change-callback updates the corresponding `OtherPlayer`'s target; and

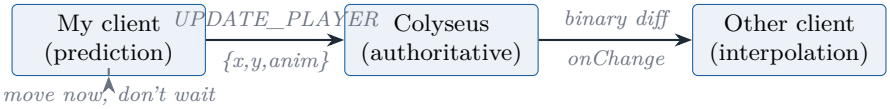


Figure 6.1. Movement synchronisation: local prediction, authoritative relay, remote interpolation. Only changed fields cross the wire.

that `OtherPlayer` interpolates toward it. A single elegant identity choice threads the whole system: the Colyseus *session id* doubles as the key in every state map *and* as the PeerJS peer id used for media, so one identifier follows a user everywhere.

6.6 Interactive Objects

The world is populated with objects that blend local responsiveness with server-authoritative shared state:

Chairs are purely local: pressing *E* snaps the avatar into a direction-specific seated pose. Because the resulting position and animation are broadcast like any movement, everyone sees you sit — but the chair itself holds no server state.

Computers are screen-sharing stations with *server-tracked occupancy*. Pressing *R* adds your session id to that computer's connected set; the change propagates to all clients, updating an "N users" label, and anyone already at that computer receives your shared screen. This makes "who is at this computer" consistent for everyone.

Whiteboards work the same way, and each carries a server-generated random id that maps to a hosted collaborative whiteboard shown in an embedded frame, so everyone at the same whiteboard shares one board.

Vending machines are a decorative touch that open an external link.

The design principle is a clean split: *who is using* a shared object is authoritative server state (consistent for all), while *what you can interact with right now* is decided by local physics (responsive, no round-trip).

6.7 Proximity Communication

The office's signature feature is that communication follows position. It is built on three mechanisms.

Proximity voice and video (PeerJS/WebRTC). When two avatars physically overlap, the game's overlap handler attempts a call. To avoid both sides dialling at once, only the peer with the greater session id initiates, after a short debounce and only if both peers have enabled their webcam and marked themselves ready; the other side auto-answers. Encrypted audio and video then flow directly peer-to-peer. When the avatars separate, the call is torn down — except inside a designated private-meeting rectangle, where the call deliberately persists so a group can gather without staying pixel-perfectly overlapped.

Screen sharing. A *second*, separate PeerJS peer (its id suffixed `-ss`) isolates screen media from webcam media. When a user shares their screen at a computer, the manager calls every user currently connected to that same computer object, and new arrivals are called as their connection propagates through the shared state; a "stop sharing" signal is relayed so remote tiles disappear cleanly.

Text chat. A sent message follows two paths from the server at once: it is appended to the synchronised, capped chat history (so late-joiners see it), and it is broadcast immediately as a transient speech bubble above the sender's avatar. The React chat panel shows the scrollback alongside synthetic "joined"/"left" system lines.

6.8 Integration with the Wider Platform

The 2D office is a Colyseus-based fork, but it does not stand alone: the backend is deliberately shaped to drive a spatial client, and a demonstration stack wires the two together. On the backend side the seams are explicit. The workspace service exposes an internal `/session-config` endpoint that returns exactly the boot payload a Colyseus room needs — the workspace, its active desks, and its active map objects (computers and whiteboards, each already carrying a stable `roomId`) — and accepts avatar `/presence` updates. The room service consumes avatar position snapshots to compute proximity voice groups (Section 5.5), and both the chat and room services bind their channels to the same workspace.

These seams are exercised concretely by a one-command **demonstration stack** (`backend/dev`). A single script brings up the infrastructure (PostgreSQL, MongoDB, Redis, RabbitMQ) in containers, starts the workspace, chat, room, and gateway services together with the Colyseus fork and the SkyOffice client, seeds two workspaces of five members each *consistently across every service*, mints a demonstration JWT, and prints a URL per user. The workspace service seeds (through a Flyway migration) a tile map with a meeting-room zone and open floor, five active desks placed near the spawn so that two users land in the same proximity voice group, and computers and whiteboards; the chat and room services seed the matching canonical channel and a shared voice room for the same members. Opening the two printed URLs in separate browser sessions lets two avatars walk the same office, meet in proximity to trigger voice, and exchange chat — all backed by the real, running microservices rather than by a static map. Deepening this binding (so that every avatar, desk, and object is served live from the workspace and every movement feeds the proximity engine in steady state) is the natural next step, identified in the future-work roadmap of Chapter 10.

CHAPTER 7

The Web Application

The web client is VIRTUAL OFFICE's primary collaboration surface: a Slack-style single-page application through which a member manages workspaces, chats in channels and direct messages, opens threads, joins room calls with live voice/video, and receives notifications. This chapter describes its architecture, its authentication and API strategy, its three-connection real-time layer, its Agora-based calling, and a feature-by-feature tour of the application.

7.1 Overview and Technology Stack

The client is a React 18 single-page application created with Create React App. It uses **MUI 7** and **Tailwind CSS** for its interface, **axios** and the browser `fetch` API for REST, **`@stomp/ stompjs`** for real-time messaging, and **`agora-rtc-sdk-ng`** for voice and video. State management is deliberately minimal: there is no Redux or global context. Component-local state is lifted into the top-level chat page and passed down by props, session and theme live in `localStorage`, and the notification socket is a module singleton. This keeps the codebase small and the data flow easy to follow.

> The web client is the Slack-style collaboration app, *not* the 2D office; the animated world (Chapter 6) is a separate client. Within the web app, the "office" metaphor is realised through workspaces and through `ROOM`-type channels that host Agora audio/video calls.

7.2 Application Structure and Routing

The application boots by mounting the router inside a strict-mode root, with an early theme bootstrap that reads the saved (or system) colour scheme and toggles a dark-mode class on the document body — the mechanism the whole app uses for theming. Routes divide into public and protected (Table 7.1); a lightweight `PrivateRoute` guard redirects to login when no token is present.

Table 7.1. Web client routes.

Path	Access	Purpose
`/login`, `/signup`	Public	Authentication
`/verify-email`	Public	OTP e-mail verification
`/chat`	Protected	The main application (workspaces, channels, rooms, DMs, threads, calls)
`/chat/join/:channelId`	Protected	Invite-link landing; joins a channel then redirects

7.3 Authentication and the API Layer

On login or registration the client stores the returned JWT in `localStorage` and navigates into the application. It reads the current user's numeric id by decoding the JWT payload on the client side — that id is sent as the `X-User-Id` header on nearly every subsequent request. Two REST conventions coexist: an **axios instance** (used for auth and profile) whose interceptors attach the bearer token and, on any `401`, clear the session and redirect to login; and **raw `fetch`** calls (used for chat, workspace, rooms, and notifications) that set the identity headers by hand.

During development, a proxy configuration doubles as the service map, forwarding each path prefix to the corresponding microservice port and, importantly, wiring the *WebSocket upgrade* for the chat and room sockets (Table 7.2). In production these resolve to the gateway.

Table 7.2. Development proxy: the frontend's view of the backend.

Path prefix	Port	Service
`/api/auth`, `/api/users`	8091	User
`/api/chat` (+WS)	8084	Chat
`/api/rooms`, `/ws/rooms`	8086	Room
`/api/workspace`	8087	Workspace
`/api/notifications`	8082	Notifications

7.4 The Real-time Layer: Three STOMP Connections

The web client maintains *three* independent STOMP connections, one to each real-time service, all following the ticket pattern of Section 4.6: before every (re)connection a fresh single-use ticket is fetched, and a five-second reconnect delay provides automatic recovery.

Chat (8084). The core connection. The message list for the active channel subscribes to `/topic/channel/{id}` and handles the `NEW_MESSAGE`, `EDIT_MESSAGE`, and `DELETE_MESSAGE` actions, merging live events with fetched history by id. The composer publishes to `/app/chat/send`. Threads use the parallel `/topic/thread/{id}`.

Rooms (8086). When the active channel is a room, the client subscribes to `/topic/room/{roomId}` for call presence (`PARTICIPANT_JOINED`/`LEFT`, `STATE_CHANGED`) and publishes its own join/leave to `/app/room/state`, so the "N in call" indicator stays live for everyone.

Notifications (8082). Implemented as a reference-counted module singleton so the whole app shares one socket; it subscribes to `/user/queue/notifications` and raises the unread badge on each `NEW_NOTIFICATION`.

Because every service uses the same `{action, payload}` envelope, each of these subscriptions is handled by the same simple pattern — switch on the action, apply the payload — which is a direct payoff of the architectural uniformity described in Chapter 4.

7.5 Voice and Video with Agora

Real-time media in the web client is room-scoped and handled by a single reusable hook that encapsulates the entire Agora engine. The flow to join a call (Figure 7.1) is:

1. The user opens a `ROOM`-type channel and clicks *Join Call*.
2. The client resolves the real room id and calls `POST /api/rooms/{id}/join`, which returns the Agora channel name and a freshly minted access token — the room service is the token server (Section 5.5).
3. The hook joins the Agora channel with that token, creates and publishes the local microphone and camera tracks, and subscribes to remote participants as they publish, rendering each into a video tile.
4. In parallel, the client broadcasts its presence over the rooms STOMP connection so other members see the participant list update.

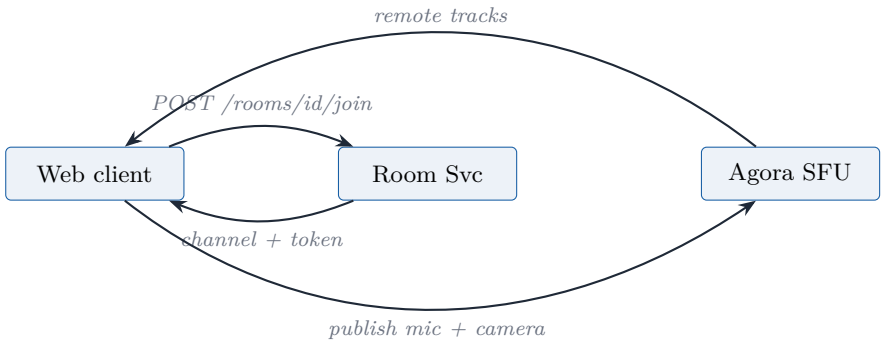


Figure 7.1. Joining a room call: the room service mints the Agora token, media flows client-to-SFU.

Toggling the microphone or camera flips the enabled state of the local tracks; leaving closes the tracks, leaves the channel, and broadcasts departure. Crucially the media never traverses Virtual Office's own servers — the backend's only role is to authorise the join by issuing a scoped token.

7.6 Feature Tour

The protected `/chat` route is the composition root and holds nearly all application state. Its responsive four-column layout — workspace rail, channel sidebar, chat area, and (when open) thread panel and member list — adapts down to a drawer-based mobile layout.

Workspaces. A vertical rail lists the user's workspaces (loaded from the workspace service); if the user has none, a default workspace is created automatically. A dialog creates new workspaces with a name, auto-generated slug, description, and timezone.

Channels, rooms, and DMs. The sidebar lists group channels, room channels, and direct messages for the active workspace, auto-provisioning a set of starter channels for a brand-new workspace. Selecting a channel loads its history and subscribes to its live topic; selecting a room additionally enables the call interface. Direct messages are opened by target user id and guaranteed unique per pair by the backend.

Messaging. The chat area renders the merged history-plus-live message stream with date dividers, resolves sender names through a client-built user map, and supports inline editing and deletion of one's own messages, replies, mentions, an emoji picker, and a

client-side message search. A "reply in thread" action opens the slide-in thread panel, which creates or loads a thread and subscribes to its topic.

Rooms and calls. For room channels, a call bar and a video grid layer the Agora experience described above on top of the same message timeline, with a live participant count driven by the rooms socket.

Notifications. A bell in the sidebar footer shows the unread count, opens a list fed by REST plus the live notifications socket, and supports mark-read and delete.

Settings and invites. A settings dialog handles avatar upload and password change; channels can be shared by a copyable invite link that lands on the `/chat/join/:channelId` route and auto-selects the joined channel.

7.7 Design Observations

Several deliberate choices are worth noting for the evaluation in Chapter 9. The ticket-based WebSocket authentication, refreshed on every reconnect, is a clean and robust solution to the browser's inability to set socket headers. The single `{action, payload}` dispatch pattern keeps the real-time code uniform across three sockets. On the other hand, the client is honestly at *prototype* maturity in places: some creation flows use native browser prompts, presence beyond "in this call" is not yet backed by a server roster, the typing indicator is sent but not yet displayed, and message attachments are rendered but not yet uploadable. These are precisely the rough edges catalogued as future work.

CHAPTER 8

The Mobile Application

The mobile client brings VIRTUAL OFFICE's messaging to phones. It is a Flutter application focused on the chat domain — channels, direct messages, threads, read receipts, typing indicators, and, importantly, push notifications that work whether the app is in the foreground, the background, or fully terminated. This chapter describes its clean-architecture layering, its Provider-based state management, its dual-transport networking, its real-time and notification stacks, and the engineering details that make it feel responsive and reliable.

8.1 Overview and Scope

The application is built with Flutter and Dart and uses a small, deliberate set of packages: **Provider** for state management, `http` for REST, `stomp_dart_client` for real-time messaging, `awesome_notifications` for local/push notifications, `shared_preferences` for persistence, and `uuid` for client-generated message ids. Its scope is intentionally the *chat* experience: the mobile app does not implement the 2D office, workspace administration, or in-app calling. This focus lets it implement the messaging domain to a high standard — optimistic delivery, reconnection catch-up, and cross-state notifications — rather than implementing everything shallowly.

8.2 Architecture

The code follows a **feature-first, clean-architecture-inspired** layering in which dependencies flow strictly downward (Figure 8.1). The *repository* is the linchpin: it is the single source of truth that merges the REST data source, the WebSocket data source, and an in-memory cache, and re-broadcasts consolidated state through its own streams that the providers consume.

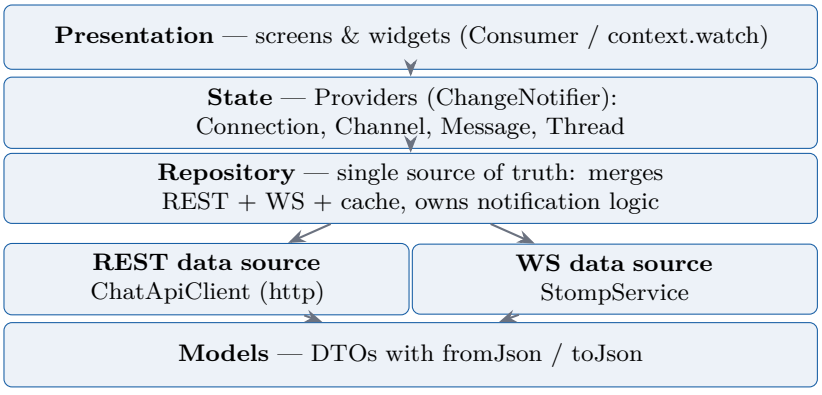


Figure 8.1. Mobile clean-architecture layering; data flows WS/REST → repository → providers → widgets.

Dependency injection is manual, through Provider's `MultiProvider`: a single root `ConnectionProvider` lazily constructs the repository (and its REST and WebSocket data

sources) on a successful connection, and hands it down to feature-scoped providers created as screens are pushed.

8.3 State Management with Provider

Five `ChangeNotifier` providers structure the application's state:

`ConnectionProvider` (root) owns the server configuration and connection status and the singleton repository. Its `connect()` normalises the URLs, runs a health check, opens the socket, loads the workspace's channels and DMs, and *auto-subscribes to all of them* so that notifications arrive app-wide even for conversations the user is not currently viewing.

`ChannelProvider` holds the channel and DM lists and per-channel unread counts, and exposes load/join/create/leave operations.

`MessageProvider` manages one open conversation: a `createdAt`-sorted message list, reply/edit state, and a debounced typing indicator; it subscribes to the repository's message and typing streams filtered to its channel and performs catch-up on reconnect.

`ThreadProvider` and **`ThreadMessageProvider`** mirror the channel and message providers for threads.

A recurring correctness theme is safe disposal: providers guard against calling `notifyListeners` after disposal and cache the repository reference so that teardown never touches a `BuildContext` that may already be unmounted.

8.4 Networking: A Dual-Transport Repository

The repository speaks to the chat service over two transports and degrades gracefully between them.

REST. The `ChatApiClient` wraps the `http` package with the identity headers (`X-User-Id`, `X-User-Role`), a ten-second timeout, and typed error handling. It covers the full chat surface: channels and DMs, message history (with `before`/`after` cursors for pagination and catch-up), send/edit/delete, threads, and read receipts.

Real-time. The `StompService` connects to `/api/chat/connect?ticket=<t>` (the same ticket pattern as the web client), auto-reconnects, and exposes the incoming message and typing events as Dart streams. It subscribes to `/topic/channel/{id}` and its typing sibling, and to the thread equivalents, and publishes sends to `/app/chat/send`.

Optimistic delivery. Sending is optimistic (Figure 8.2): the repository immediately inserts a temporary message keyed by a generated `clientMessageId`, attempts delivery over the WebSocket, and *falls back to a REST `POST`* if the socket send throws. When the server's authoritative echo arrives — matched by that same `clientMessageId` — it replaces the placeholder; on total failure the optimistic message is rolled back. On reconnection the repository fetches any messages that arrived while it was offline (everything `after` the last known id), so no message is missed.

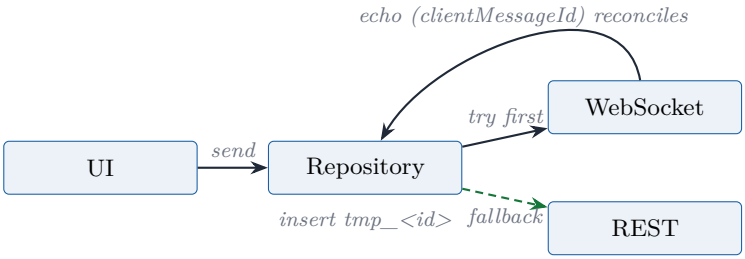


Figure 8.2. Optimistic send with WebSocket-first, REST-fallback, and echo reconciliation.

8.5 Identity on Mobile

The mobile app takes a pragmatic approach to identity suited to a development and demonstration client: rather than a full login screen, a *connection screen* lets the user enter the server URLs, a numeric user id, and a role, which are persisted with `shared_preferences` and sent as the `X-User-Id` and `X-User-Role` headers — exactly the headers the gateway would otherwise inject after validating a JWT. The WebSocket still uses the proper single-use ticket handshake. This mirrors, on the client, the platform's server-side trust model (Chapter 4); wiring a real JWT login is a straightforward extension, since the backend already issues and validates the token.

8.6 Notifications Across App States

The notification system is the mobile client's most sophisticated piece. Built on `awesome_notifications`, it defines four styled channels (channel messages, direct messages, thread replies, and system notices) and, critically, registers its lifecycle handlers as static, entry-point-annotated functions so they execute even when the app is *backgrounded or terminated*.

The decision of *whether* to raise a notification lives in the repository: a notification fires only when a message is not from the current user *and* the user is not currently viewing that channel or thread (tracked by "active" sets updated as screens are entered and left). Each notification embeds a payload (channel id, thread id, DM flag, message id) so that tapping it deep-links to the right conversation. A small navigation bridge makes this reliable across all three app states: if the app is running, it navigates immediately; if it was terminated, the target route is stored and replayed once the app's navigator is ready on the first frame.

8.7 Data Models and Bootstrapping

The model layer is a set of null-safe DTOs with `fromJson`/`toJson` — `ChannelResponse`, `MessageResponse` (with `deleted`, `threadId`, `replyToId`, `mentions`, and `clientMessageId`), `ThreadResponse`, a generic `PaginatedResponse`, and the typing and error envelopes — mirroring the chat service's contract. Application startup initialises the notification service (its channels, permissions, and background handlers) *before* the app runs, then mounts the root provider and a `MaterialApp` that holds a global navigator key (so the notification bridge can navigate from outside the widget tree) and routes to the connection screen.

8.8 Engineering Highlights

For the evaluation in Chapter 9, the mobile client demonstrates several production-grade patterns worth naming: optimistic UI with `clientMessageId` reconciliation; dual-transport

delivery with graceful degradation; reconnection resilience with message catch-up; disposal safety; and correct notification behaviour across foreground, background, and terminated states. Its main limitation is one of *breadth* rather than depth: it covers only the chat domain, leaving workspace administration, the 2D office, and in-app calling to the web and game clients.

CHAPTER 9

Testing, Deployment, and Evaluation

This chapter describes how Virtual Office is tested and run, and then evaluates the system against the requirements of Chapter 3. In keeping with the rest of the book, the evaluation is honest: it records both what works end-to-end and what remains partial or unimplemented, so the account is a faithful picture of a substantial but still-evolving system.

9.1 Testing Strategy

Virtual Office is tested at several levels, though not uniformly across services.

Unit tests exercise individual components in isolation — service methods, mappers, and validation logic. The workspace service is the most rigorously tested: its build enforces a 90% line-coverage threshold through JaCoCo, so its business logic (role checks, invitation lifecycle, optimistic layout updates) is well covered.

Integration tests verify a service together with its real infrastructure. The Mongo/Redis-backed services (chat, room, notifications) use **Testcontainers** to spin up genuine MongoDB, Redis, and RabbitMQ instances inside the test run, so that database queries, index constraints, Redis scripts, and message-broker round-trips are exercised against the real technologies rather than mocks. The workspace service uses Failsafe-driven integration tests against PostgreSQL with Flyway-migrated schemas.

Contract documentation takes the place of some cross-service testing: each implemented service publishes an OpenAPI specification (rendered through a Swagger UI in the workspace service), which fixes the request/response contract that clients and other services code against.

Manual and exploratory testing covers the real-time and multi-client behaviours that are hardest to automate — proximity calls, presence updates, notification delivery across app states — and is supported by the demonstration stack described next.

9.2 Deployment and the Demonstration Stack

Every service ships with the container definitions for its own dependencies, and Spring Boot's Docker Compose integration allows a service to bring those dependencies up automatically on start. Service locations are externalised as environment variables, so the same artefacts run unchanged from a developer laptop to a container cluster; in a clustered deployment the gateway is the only publicly exposed process, with everything else on an internal network.

The clearest evidence that the platform runs as an integrated whole is the **one-command demonstration stack**. A single script:

1. starts the shared infrastructure (PostgreSQL, MongoDB, Redis, RabbitMQ) in Docker;
2. launches the workspace, chat, room, and gateway services, together with the Colyseus fork and the SkyOffice 2D client;

3. seeds *two workspaces of five members each*, consistently across every service — the workspace service seeds (via a Flyway migration) a tile map with a meeting-room zone, five desks near the spawn point, and computers and whiteboards; the chat service seeds the canonical channel with those members and some messages; the room service seeds a shared voice room;
4. mints a demonstration JWT and prints a ready-to-open URL for each of two users.

Opening the two URLs in separate browser sessions places two avatars in the same seeded office, close enough to fall into one proximity voice group, sharing a channel and a voice room — so a reviewer can, in a single command, walk two avatars together to trigger proximity voice and exchange chat, all backed by the real microservices. All seeders are idempotent, so the stack can be restarted freely. This stack is both a testing aid and the canonical way to demonstrate the system.

9.3 Observability: Metrics and Dashboards

Running a distributed system is only manageable if its runtime behaviour is visible. Beyond the per-service health endpoints and application logs, VIRTUAL OFFICE exposes a **metrics** pipeline built on the standard Spring Boot stack and visualised in Grafana. It covers the five implemented services — **user** (`:8091`), **workspace** (`:8087`), **chat** (`:8084`), **room** (`:8086`), and **notifications** (`:8082`).

9.3.1 The Metrics Pipeline

Every service already depends on Spring Boot Actuator; adding a Micrometer Prometheus registry turns each one into a metrics source. Micrometer instruments the application automatically — HTTP server timings, JVM and process internals, and the datastore and connection-pool clients — and Actuator publishes the result as a Prometheus-format text endpoint at `/actuator/prometheus`. A single **Prometheus** server periodically *scrapes* that endpoint on each service and stores the samples as time series; **Grafana** then queries Prometheus with PromQL and renders the dashboards. The flow is pull-based and one-directional, as shown in Figure 9.1.

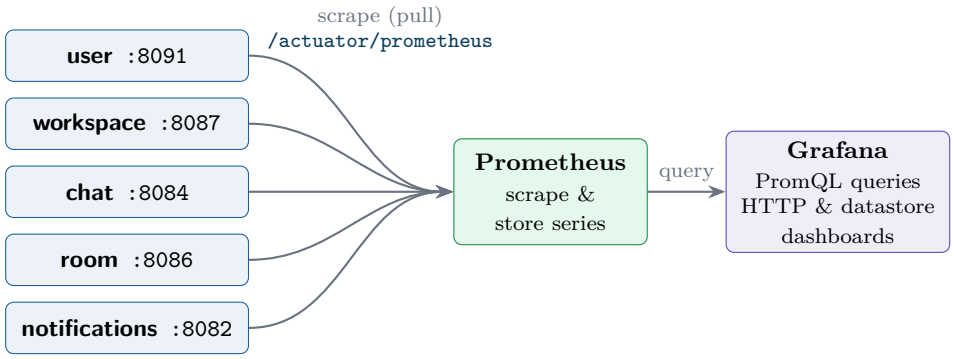


Figure 9.1. The metrics pipeline: each implemented service exposes a Prometheus endpoint; one Prometheus server scrapes all five and stores the series, and Grafana queries it to render the HTTP and datastore dashboards.

Exposing the endpoint is a small, uniform change per service: the `micrometer-registry-prometheus` dependency on the classpath, and the endpoint added to the Actuator exposure list.

```
management:
```

```
  endpoints:
    web:
      exposure:
        include: health, info, prometheus
  metrics:
    tags:
      application: ${spring.application.name}
```

Listing 9.1. Enabling the Prometheus endpoint in each service (`application.yml`).

Prometheus is then pointed at the five services with a single scrape job.

```
scrape_configs:
  - job_name: virtual-office
    metrics_path: /actuator/prometheus
    static_configs:
      - targets: ["user:8091", "workspace:8087", "chat:8084",
                  "room:8086", "notifications:8082"]
```

Listing 9.2. The Prometheus scrape job targeting the five services (`prometheus.yml`).

Because scraping is pull-based, the services need know nothing about Prometheus — they simply expose the endpoint, so adding or removing a target is a change to Prometheus's configuration alone.

9.3.2 What the Dashboards Show

The Grafana dashboards concentrate on the two signals that matter most for this system's health: request behaviour at the edge of each service, and the state of the datastores behind them.

HTTP request rate, latency, and errors are derived from Micrometer's `http.server.requests` timer, which records every handled request tagged by `method`, `uri`, and response `status`. From it the dashboards plot per-service throughput, latency percentiles (the timer captures a distribution, so p95/p99 are available), and the error rate as the share of `4xx`/`5xx` responses — the quickest way to see a service slowing down or failing a dependency.

Datastore health is read from the persistence clients each service already uses. The two SQL services surface **HikariCP** connection-pool gauges — active, idle, and pending connections and acquisition time — for **user**'s MySQL and **workspace**'s PostgreSQL, making pool exhaustion or slow queries visible before they surface as request errors; the Mongo/Redis services (**chat**, **room**, **notifications**) surface their MongoDB and Redis client metrics on the equivalent panels.

Because every service is scraped uniformly, a dashboard can be read per-service or aggregated across the fleet, so a reviewer can watch the whole backend from one screen while exercising the demonstration stack of the previous section.

Scope. Observability here is metrics-and-dashboards for the five implemented services; it deliberately does not yet include the API gateway or the unimplemented **calendar**/**tasks** scaffolds, and it stops short of distributed tracing and alerting. The Prometheus endpoint also carries the standard JVM and process metrics even though the current dashboards focus on HTTP and datastore health, so those panels can be added without any change to the services.

9.4 Evaluation against Requirements

Table 9.1 assesses the functional requirement groups of Section 3.2. The status column distinguishes fully working end-to-end capabilities from those that are partial (implemented on some clients or with known gaps) or unimplemented.

Table 9.1. Evaluation of functional requirement groups.

Requirement group	Status	Notes
Identity & accounts (FR-01–07)	Complete	Registration, login, JWT, OTP, profile, photo; events emitted.
Workspaces, desks, teams (FR-08–15)	Complete	Full PostgreSQL/Flyway service with roles, invitations, map layout, and the internal session API.
Messaging (FR-16–24)	Complete	Channels, DMs, threads, edits, mentions, read receipts, idempotency, event-driven channel provisioning; on both web and mobile.
Meeting rooms & proximity voice (FR-25–28)	Complete (backend)	Rooms, presence, Agora tokens, and the deterministic proximity-voice engine all implemented; consumed by the web client.
2D office (FR-29–33)	Complete (client)	Movement, avatars, interactive objects, proximity media, in-world chat; integrated in the demo stack.
Notifications (FR-34–37)	Complete	E-mail and in-app paths, real-time push, idempotency. The `TASK_ASSIGNED` consumer awaits its producer.
Web client (FR-38)	Complete	Full collaboration surface; some prototype-grade UX.
Mobile client (FR-39)	Partial	Chat domain implemented to a high standard; no office/workspace admin/calling.
Real-time auth (FR-40)	Complete	Ticket handshake and auto-reconnect on all clients.
Tasks & calendar	Not implemented	Scaffolds only; the notification consumer for tasks exists.

9.5 Non-functional Evaluation

Against the non-functional targets of Section 3.3, the design choices documented throughout the book meet the important ones:

- **Performance (NFR-01)** is addressed structurally: avatar movement uses client-side prediction with remote interpolation, and Colyseus transmits only changed fields; chat delivery is push-based over persistent sockets.
- **Scalability (NFR-02)** is met by keeping media off the application servers entirely — peer-to-peer in the office and an external SFU for rooms — so server load does not grow with audio/video volume.
- **Security (NFR-03)** is enforced at the edge: the gateway validates the JWT once, strips client-supplied identity headers, and blocks internal endpoints, while internal calls carry a separate shared token.

- **Reliability and consistency (NFR-04, 05)** come from durable, idempotent, retried events and the fail-closed internal calls, giving eventual consistency that clients tolerate.
- **Maintainability and portability (NFR-06, 07)** follow from per-service data ownership and the container definitions and demo stack.

The principal non-functional gaps are operational hardening rather than design flaws: secrets (the JWT signing key, mail credentials, the Agora certificate, and the internal token) are currently committed as configuration defaults and should move to a secret manager; platform-level role-based access control is not yet modelled (the gateway assigns every user the same platform role, with real authority resolved per workspace); and observability, while it now includes Prometheus metrics and Grafana dashboards for the five implemented services (Section 9.3), does not yet extend to the gateway, distributed tracing, or alerting.

9.6 Limitations

Consolidating the honest notes made throughout the book, the current limitations are:

1. The **tasks** and **calendar** services are unimplemented scaffolds, and the **shared library** is an empty placeholder.
2. The **mobile** client covers only chat.
3. The **2D office** runs as an integrated demo but is not yet driven fully live from the workspace service in steady-state operation.
4. Several **secrets** are configuration defaults rather than injected at deploy time.
5. Some **web UX** is prototype-grade, and a few features (presence roster, typing display, attachment upload) are partially wired.

None of these undermines the core result: an integrated, multi-client, real-time collaboration platform whose central capabilities — identity, workspaces, messaging, rooms, proximity communication, and notifications — work together end-to-end. The next chapter reflects on that result and lays out the roadmap these limitations imply.

CHAPTER 10

Conclusion and Future Work

10.1 Summary

This book has documented the design and implementation of Virtual Office, an all-in-one, spatial team-collaboration platform. Starting from a concrete problem — the fragmentation of remote-work tools and the loss of the *presence* and *spontaneity* of a physical office — the project set out to give a distributed team a single *place* rather than a collection of applications, and to build it as a properly architected, multi-client, real-time system.

The result is a platform with real breadth. A 2D virtual office, built on Phaser and an authoritative Colyseus server, lets users appear as avatars, move around a shared world, interact with computers and whiteboards, and fall into proximity-based voice, video, and screen sharing. A React web client delivers a full Slack-style collaboration surface — workspaces, channels, direct messages, threads, room calls over Agora, and live notifications. A Flutter mobile client brings the messaging experience to phones with optimistic delivery and cross-state push notifications. Underneath, a backend of cooperating Spring Boot microservices — gateway, user, workspace, chat, room, and notifications — handles identity, spatial and organisational state, real-time messaging, and event-driven side effects, over a polyglot mix of MySQL, PostgreSQL, MongoDB, Redis, and RabbitMQ.

10.2 Achievements

Measured against its objectives (Section 3.2), the project delivered:

- A working **real-time multiplayer office** with client-side prediction, remote interpolation, server-authoritative interactive objects, and proximity peer-to-peer media.
- A complete **messaging system** — channels, DMs, threads, mentions, edits, soft deletion, read receipts, typing, and idempotent delivery — realised on both web and mobile.
- A rich **workspace and identity layer** modelling companies, desks, teams, roles, invitations, and an editable 2D map, with centralised authentication and an internal API designed to drive the spatial client.
- A **room and proximity-voice service** that mints media tokens and computes voice groups from avatar positions without ever carrying media.
- A **unified notification service** spanning transactional e-mail and real-time in-app delivery.
- A coherent set of **cross-cutting patterns** — edge-validated identity with header propagation, ticket-based WebSocket authentication, the uniform `{action, payload}` envelope, and idempotent event-driven decoupling — applied consistently across five services and three clients, and demonstrable end-to-end through a one-command stack.

Beyond the artefact, the project's value is the demonstration that four very different concerns — a real-time multiplayer game, a Slack-style chat system, a microservices identity/workspace layer, and peer-to-peer media — can be integrated into one coherent product.

10.3 Lessons Learned

Several engineering lessons recur through the work. *Uniform patterns pay compound dividends*: because every real-time service used the same ticket handshake and the same message envelope, each new client needed only one connection recipe and one dispatch loop. *Asynchronous decoupling buys resilience but costs immediacy*: event-driven binding between services survives outages and avoids tight coupling, at the price of eventual consistency that clients must be designed to tolerate. *Security is easiest when it lives in one place*: validating identity once at the gateway, and having every service trust the result, removed authentication logic from five services at a stroke — provided the edge is disciplined about stripping spoofable headers. And *polyglot persistence rewards fitting the store to the data*: relational integrity for workspaces, documents for messages, and Redis for ephemeral presence and tickets each simplified their service.

10.4 Future Work

The limitations catalogued in Chapter 9 map directly onto a roadmap, ordered roughly by leverage.

Complete the tasks and calendar services. These are the largest functional gaps. The notification service already *consumes* `TASK_ASSIGNED` events, so implementing the tasks service — task creation, assignment, progress, and deadlines — immediately lights up in-app and push task notifications end-to-end. The calendar service (scheduling, meeting invites) would similarly emit events consumed by notifications and could bind meetings to rooms.

Fully integrate the 2D office with the backend. The demonstration stack already runs the office against the seeded services; the next step is to drive it *live* in steady state — boot each Colyseus room from the workspace service's `/session-config`, stream avatar positions to the room service's proximity engine, and reflect desk presence and status — so that the spatial world and the collaboration backend are one system rather than two that meet in a demo.

Broaden the mobile client. Extend it beyond chat to workspace membership, room presence, and eventually a mobile view of the office, and add a real JWT login on top of the existing header-based identity.

Harden security for production. Move all secrets (JWT signing key, mail credentials, Agora certificate, internal token) into a secret manager; introduce platform-level role-based access control alongside the existing per-workspace roles; and enforce the authorisation checks currently trusted to the client.

Strengthen reliability and observability. Add dead-letter queues and a reaper for orphaned presence, and build on the existing Prometheus/Grafana metrics (Section 9.3) with distributed tracing, alerting, and centralised logging so the real-time flows can be observed in production.

Consolidate shared code. Populate the shared library with the common DTOs, user-context helpers, and exception handling that each service currently re-implements, reducing drift.

Polish the clients. Finish the partially wired web features (a server-backed presence roster, a visible typing indicator, attachment upload) and replace prototype-grade interactions with production UX.

10.5 Closing Remarks

Virtual Office set out to reproduce, online, the two things a physical office gives a team for free — knowing who is around, and being able to simply walk over and talk — and to fold the rest of the working day into the same place. The delivered system does this at its core: avatars share a live space, conversations begin by proximity, and persistent messaging, rooms, workspaces, and notifications surround that space through a web, a mobile, and a game client over a coherent microservices backend. What remains is breadth and hardening rather than a change of direction. As a piece of engineering, the project stands as a demonstration of how a real-time, spatial, multi-client collaboration platform can be designed and built — and as a foundation ready to be carried further.